



M256 Unit 10
UNDERGRADUATE COMPUTING

Software development with Java



Coding and testing

Unit **10**

This publication forms part of an Open University course M256 *Software development with Java*. Details of this and other Open University courses can be obtained from the Student Registration and Enquiry Service, The Open University, PO Box 197, Milton Keynes MK7 6BJ, United Kingdom: tel. +44 (0)845 300 60 90, email general-enquiries@open.ac.uk

Alternatively, you may visit the Open University website at <http://www.open.ac.uk> where you can learn more about the wide range of courses and packs offered at all levels by The Open University.

To purchase a selection of Open University course materials visit <http://www.ouw.co.uk>, or contact Open University Worldwide, Michael Young Building, Walton Hall, Milton Keynes MK7 6AA, United Kingdom for a brochure. tel. +44 (0)1908 858793; fax +44 (0)1908 858787; email ouw-customer-services@open.ac.uk

The Open University
Walton Hall
Milton Keynes
MK7 6AA

First published 2007. Second edition 2008.

Copyright © 2007, 2008 The Open University.

All rights reserved. No part of this publication may be reproduced, stored in a retrieval system, transmitted or utilised in any form or by any means, electronic, mechanical, photocopying, recording or otherwise, without written permission from the publisher or a licence from the Copyright Licensing Agency Ltd. Details of such licences (for reprographic reproduction) may be obtained from the Copyright Licensing Agency Ltd, Saffron House, 6–10 Kirby Street, London EC1N 8TS; website <http://www.cla.co.uk>

Open University course materials may also be made available in electronic formats for use by students of the University. All rights, including copyright and related rights and database rights, in electronic course materials and their contents are owned by or licensed to The Open University, or otherwise used by The Open University as permitted by applicable law.

In using electronic course materials and their contents you agree that your use will be solely for the purposes of following an Open University course of study or otherwise as licensed by The Open University or its assigns.

Except as permitted above you undertake not to copy, store in any medium (including electronic storage or use in a website), distribute, transmit or retransmit, broadcast, modify or show in public such electronic materials in whole or in part without the prior written consent of The Open University or in accordance with the Copyright, Designs and Patents Act 1988.

Edited and designed by The Open University.

Typeset by The Open University.

Printed and bound in Malta by Gutenberg Press.

ISBN 978 0 7492 5483 4

CONTENTS

1	Introduction	5
1.1	The role of the implementation model	6
1.2	The aims of this unit	6
1.3	Studying this unit	7
1.4	The projects in this unit	7
2	Testing	8
2.1	What does testing involve?	8
2.2	Choosing test cases	9
2.3	Testing code	12
3	Beginning implementation	14
3.1	Instance variable declarations	14
3.2	Constructors	16
3.3	Getter methods for attributes	17
3.4	The first code iteration	20
4	From implementation model to code	23
4.1	Using dynamic models	23
4.2	Examining method code	26
5	Testing a method	31
5.1	Specifying test cases	31
5.2	Writing and running tests	34
6	Implementing a use case	38
6.1	Planning the implementation	38
6.2	Coding and testing the <code>contains()</code> method	40
6.3	Coding and testing the <code>recordTreatmentBy()</code> method	44
6.4	Coding the <code>recordTreatment(aPatient, aDoctor)</code> method	47
7	The Hospital core system implementation	49
8	Summary	51
	Glossary	52
	References	54
	Acknowledgements	54
	Index	55

M256 COURSE TEAM

Affiliated to The Open University unless otherwise stated.

Sarah Mattingly, Course Chair and Author

Lindsey Court, Author

Marion Edwards, Software Developer

Rob Griffiths, Critical Reader

Benedict Heal, Critical Reader

Simon Holland, Author

Barbara Poniatowska, Course Manager

Barbara Segal, Author

Rita Tingle, Author

Richard Walker, Author and Critical Reader

Robin Walker, Author and Critical Reader

Ray Weedon, Academic Editor

Ray Welland, External Assessor, University of Glasgow

Julia White, Course Manager

Ian Blackham, Editor

Phillip Howe, Composer

Callum Lester, Software Developer

Andy Seddon, Media Project Manager

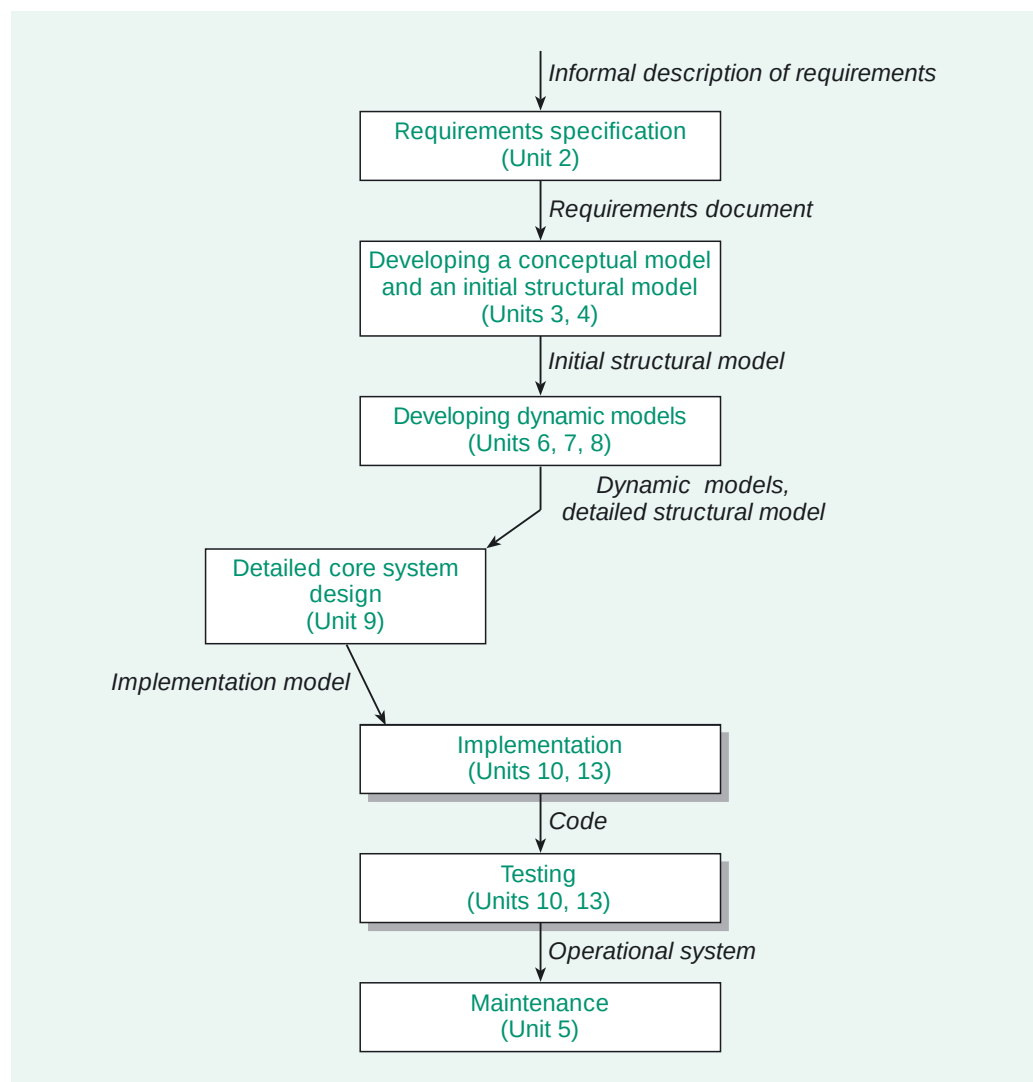
Andrew Whitehead, Graphic Artist

Thanks are due to the Desktop Publishing Unit, Faculty of Mathematics and Computing.

1 Introduction

In *Unit 9* you learnt about the kinds of detailed design decision that may be involved in completing the design phase of a software development project, and which result in the completion of the system's implementation model. In this unit you will learn about the role of the implementation model in the next phases: *coding (implementation)* and *testing*.

Figure 1 shows the place of these phases in the development process as introduced in M256.



For simplicity, Figure 1 shows the direct input to testing being code alone; in fact the inputs to the kinds of testing that you will learn about in this unit are both the code and the implementation model.

Unit 13 will look at implementing a graphical user interface.

Figure 1 The place of coding and testing in the software development process

1.1 The role of the implementation model

Recall that an implementation model has:

- ▶ a *structural part*, which is the final result of the evolution of the initial structural model and contains a class diagram, class descriptions and statements of invariants;
- ▶ a *dynamic part*, which contains dynamic models (walk-throughs and sequence diagrams) illustrating the actions and interactions that should take place within the system for each use case.

In a large-scale development, the people involved in producing an implementation model (the designers) may not be the same people involved in coding and testing (programmers and testers). Thus the implementation model is a key element in the development process. It records the decisions of the designers and is handed over to the programmers and testers, forming a bridge between design and coding/testing.

In this unit, you will learn, using the Hospital System example, how to use an implementation model as a blueprint for coding a core system. You will also learn about some key features of test-driven development, in which tests are written based on the implementation model, and then the code is written that will satisfy the tests.

You will, we hope, appreciate from your study of this unit that at this stage of development much of the hard work in developing the system has in fact already been done. With a methodical approach, writing code based on an implementation model is usually a straightforward process. Writing tests is a more subjective matter and may not be so straightforward: what constitutes an appropriate set of tests for a system is debatable, and in this unit we aim to give only a flavour of the tests that would be carried out. In a large-scale project, dedicated software testers may play a significant part, from advising programmers on what tests should be applied to each small piece of code, to testing the complete operational system.

Throughout M256 we have stressed the importance in object-oriented software development of the concept of a use case: a specific statement of a task required of the system. In coding and testing, use cases continue to have a pivotal role. You will see that we build up the system incrementally, concentrating at any one time on what is required for a single use case.

1.2 The aims of this unit

In this unit we aim to:

- ▶ present an overview of testing, describing its purposes and the different forms it can take (Section 2);
- ▶ explain how a developer might start to implement a system (Section 3);
- ▶ show how to use an implementation model as a blueprint for coding methods (Section 4);
- ▶ explain what is involved in specifying test cases based on a method specification and what is involved in writing the corresponding tests, and give you practice in using JUnit to run tests (Section 5);
- ▶ give you practice in coding and testing the methods required for a use case (Section 6);
- ▶ enable you to examine parts of a complete implementation of the Hospital core system (Section 7).

1.3 Studying this unit

This unit contains a number of SAQs and paper-based exercises, which are designed to reinforce your understanding of the concepts presented. There are also several extensive practical exercises, where you will examine and modify projects supplied to you. All these SAQs and exercises form an important part of the teaching strategy and you should work through them as they arise, and read the solutions provided before moving on.

Section 2 is a short, reading-based section. Sections 3, 4 and 5 involve practical exercises, but are relatively straightforward in content. Section 6 is likely to take more time than the other sections, since it involves substantial exercises (both paper-based and practical). Section 7 is comprised of a short practical exercise.

You are expected to have worked through Subsections 3.3, 3.4 and Section 6 of the *NetBeans Guide* before commencing this unit. You may well find it helpful to refer again to the *NetBeans Guide* as you work through the practical exercises here.

You will also need to refer to the *Handbook* during your study of this unit.

1.4 The projects in this unit

In the practical exercises in this unit, you will be required to work with a number of supplied NetBeans projects, each of which consists of an implementation of some part of the Hospital System. You should note that any attempt to run one of these projects will fail since it has no `main()` method. The `main()` method will be implemented in a class of the graphical user interface package, to which we turn in *Unit 13*.

2 Testing

In *Unit 1*, testing was introduced as referring to the activities that take place at each phase of development to ensure that the phases of development are consistent and complete with respect to each other, and also consistent and complete with respect to the requirements. Then, in *Unit 2*, you met the concept of *acceptance tests*, which are applied by the client to the operational system to confirm that it meets their needs. Broadly, the purpose of testing is to check that a software system carries out the right tasks, and that it carries them out reliably.

In this section you will learn:

- ▶ what testing involves;
- ▶ how to choose a simple set of test cases;
- ▶ about different types of test.

2.1 What does testing involve?

Verification and validation

As the *Unit 1* definition given at the beginning of this section implies, testing can be partitioned into two categories.

- 1 **Verification** This involves checking that all aspects of the system and its project documentation – e.g. requirements document, designs, implementation model and the code itself – are consistent and complete with respect to each other; in other words, verification addresses the question: ‘Has the product been built right?’
- 2 **Validation** This involves checking that all aspects of the system and its project documentation are consistent with the customer’s requirements; in other words, validation addresses the question: ‘Has the right product been built?’

SAQ 1

Unit 2 introduced the idea of acceptance tests for each use case of a system; the client applies these to the operational system to confirm that it meets their needs. In which category of testing – verification or validation – does acceptance testing lie?

ANSWER.....

Acceptance testing checks that the system is consistent with the customer’s requirements, so it is validation.

In general, testing checks not only the system itself, but also its project documentation. This means that testing involves checking that all models created during development – the conceptual model, the initial structural model, the implementation model, etc. – represent the user requirements and are consistent with each other. However, techniques involved in testing models are outside the scope of this course. We will concentrate on tests that can be applied once the system is being implemented; so in the context of this course, a **test** will be the activity of checking that a specific aspect of the code is consistent with the implementation model, and hence correctly represents the designs. It is important to note that testing should not only check that the code does

what it is supposed to, but should also check that it doesn't do anything extra that it is *not* supposed to – a point which is often overlooked.

Testing should uncover as many software defects (or **bugs**) as possible with a manageable amount of effort and over a realistic time span. To define each test, a **test case** is drawn up, which specifies the following elements:

- ▶ the data that will be used during the test in order to exercise the system, that is, the state of the relevant parts of the system before the test (the **fixture**) along with any input data;
- ▶ the expected result of the test, that is, the expected state of the relevant parts of the system after the test, along with any expected output data.

Test-driven development

In **test-driven development**, first, test cases are specified based on the specification of the aspect of the system under test, as set out in the implementation model, and then tests based on the test cases are written.

Next the code is written, and the tests are then run. If the result does not meet expectations, then a bug has been uncovered, and it is fixed before proceeding. Code is thus developed in small increments, with repeated code–test cycles.

This approach has several benefits, notably:

- ▶ it helps identify where problems are occurring so that they can be solved as they arise;
- ▶ it engenders confidence in the system as it is being built up;
- ▶ it is generally found to lead to a lower incidence of bugs in the operational system.

2.2 Choosing test cases

Choosing an appropriate set of test cases is a vital task, but often not a straightforward one. Adequate coverage is important: enough of the system must be tested under a sufficiently wide range of conditions to generate confidence in the quality of the system delivered to the customer.

Consider a system that will verify a password consisting of up to (and including) 10 alphanumeric characters entered by the user. The number of possible entries to be checked is huge – over one thousand million million – so examining them all would easily exceed the time and budget for the project. At a million tests per second, an exhaustive set of tests would take over one hundred years, and that is only to test the password-checking aspect of the system, never mind the rest!

Clearly, then, no set of test cases can guarantee that software is entirely free of defects. Instead, the purpose of testing is to gain confidence that the quality of the software is appropriate, and is delivered fit for its purpose. Of course, there needs to be a balance between completeness and feasibility of the tests: the time and effort spent testing must be realistic and manageable. Hence it is important to select a relatively small set of test cases which are in some way representative.

Living with bugs

The following is an excerpt from an article by Eric Sink, developer at the software company SourceGear.

The world's six billion people can be divided into two groups: group one, who know why every good software company ships products with known bugs; and group two, who don't. Those in group 1 tend to forget what life was like before our youthful optimism was spoiled by reality. Sometimes we encounter a person in group two, a new hire on the team or a customer, who is shocked that any software company would ship a product before every last bug is fixed.

Every time Microsoft releases a version of Windows, stories are written about how the open bug count is a five-digit number. People in group two find that interesting. But if you are a software developer, you need to get into group one, where I am. Why would an independent software vendor – like SourceGear – release a product with known bugs? There are several reasons:

- We care about quality so deeply that we know how to decide which bugs are acceptable and which ones are not.
- It is better to ship a product with a known quality level than to ship a product full of surprises.
- The alternative is to fix them and risk introducing worse bugs.

Sink (2006)

Successful testing, then, requires skill in devising test cases which are *likely* to detect bugs. How to do this is not a precise science; in fact it is an extensive and active area of academic research, which you are not expected to grasp in this course. In one unit, we can give only a flavour of what is involved, and the tests that you will perform will be very simple.

A simple testing strategy

The testing that you will perform in this unit will be **black box testing**, where each method is tested against its specification in the implementation model, without any examination of code (as opposed to **white box testing**, where the code is examined).

SAQ 2

Checking that a method matches its specification in the implementation model falls into which category of testing – verification or validation?

ANSWER.....

Checking that a method matches its specification is verification as it involves verifying that these elements of project documentation are consistent. In other words, it is checking that the method is 'right' with respect to its specification (that is built 'right'): it is not checking that the specification is 'right' (that is the 'right' product).

We will now introduce some factors that will be taken into account when devising test cases later in this unit. These factors are not comprehensive: they are merely representative of the kinds of factor that would be relevant in a real project.

It is clearly important to test each distinct specified behaviour of the software. For example, the coordinating method for the *Admit Patient* use case is specified as follows.

```
public Ward admit(Name aName, Sex aSex, M256Date aDate, Team aTeam)
    Post-condition: if there is no Ward object of the appropriate type with at least one
    free bed then null is returned.
    Otherwise a new Patient object, aPatient, is created with the supplied
    attribute values, and age according to aDate, and:
    (i) aPatient is linked to aWard, where aWard is a Ward object of the
        appropriate type with the greatest number of free beds, and
        numberOfFreeBeds of aWard is decremented;
    (ii) aPatient is linked to aTeam;
    (iii) aPatient is linked to the ConsultantDoctor object that is linked to aTeam;
    (iv) aWard is returned.
```

The specification makes clear that what the method achieves (its behaviour) should be different in two distinct situations, as follows, each of which should be tested:

- ▶ there is no appropriate ward available for the patient;
- ▶ there is an appropriate ward available for the patient.

In addition, for each specified aspect of the behaviour there may be different classes of objects involved, which may be treated differently by the code, and therefore the method will require testing with each combination of objects. Similarly, the objects involved may exist in different states – as these states may each be treated differently by the code, further testing will be needed. For example, in the case where there is an appropriate ward available, it should be checked that the method gives the specified results for both male and female patients in case it handles such cases differently. An example involving different classes of object might arise in a method involving *Doctor* objects, where it could be necessary to test for objects of the subclasses *JuniorDoctor* and *ConsultantDoctor*.

It is also standard practice to test around each **boundary**, loosely defined as a point at which the software's specified behaviour changes. Around boundaries, it is common for input values to be treated incorrectly, perhaps because of a comparison being incorrectly coded; for example, the programmer might have used `<` instead of `<=`.

SAQ 3

Give an example of an input at a boundary for the password example given above.

ANSWER.....

Since valid passwords must have 10 characters or fewer the system would have to respond differently to inputs of length 11 or more and inputs of length 10 or less: its behaviour should change. So, any input of exactly 10 characters, 'abcde12345' for example, is at a boundary. (From the perspective of what range of values should be *rejected*, a password of exactly 11 characters could also be considered to be at the boundary.)

Boundaries can be relevant to a method when it involves a collection, which might need to be tested for being empty (the lower boundary) and for being full (the upper boundary).

As an example, consider the following specification of the coordinating method for the *List Team's Patients* use case.

```
public Map<Patient, Ward> getPatientsAndWards(Team aTeam)
```

Post-condition: returns a map of pairs of `Patient` and `Ward` objects, such that for each `Patient` object `aPatient` linked to `aTeam`, the map contains the key-value pair (`aPatient`, `aWard`), where `aWard` is linked to `aPatient`.

It would be sensible to test this method in situations where the team has no patients (i.e. the `Team` object is linked to no `Patient` objects) in case the method behaves incorrectly at this extreme.

2.3 Testing code

In testing code, the following phases are commonly identified:

- ▶ unit testing;
- ▶ integration testing;
- ▶ regression testing;
- ▶ system testing.

Unit testing

Unit testing is the testing of individual units of software in the system. In object-oriented development the smallest unit of testing is usually considered to be a method (though in some contexts the unit is a class – see below).

In order to test a method, normally an object of the class will be created. Typically there will be several tests for each method, as described above.

As you have seen in the *NetBeans Guide*, JUnit provides a framework for testing in Java. A **test method** houses the code for a particular test of a particular method. The various test methods of a class are gathered together in one **test class** to be run as a single unit. Hence in contexts such as this, a class rather than a method may be considered the unit of testing.

It is easy to see that none but the simplest methods can be tested in complete isolation from the rest of the system. A method may have an argument, which is an object of another class being developed, for example; or another method may be required to put an object involved into a particular state prior to the test, in order to establish the fixture or input data. This means that unit testing almost inevitably leads straight into integration testing.

Integration testing and regression testing

Integration testing is usually defined as checking that problems do not arise from the interaction of a system's components. In *Unit 5*, we described a software component as a piece of software with a well-defined purpose that can be combined with other pieces of software to construct a larger system.

SAQ 4

What elements of the Hospital System could be classed as components?

ANSWER.....

Each class can be thought of as a small component of the system, but there are also bigger components. The core system, `hospitalcore`, which will be implemented as a Java package, is a component, as is the graphical user interface part of the system. The `m256people` component, which again will be implemented as a Java package, is a component that may be useful to other systems outside the Hospital System.

So, whenever code is implemented that involves the interaction between two or more classes of object under development, integration testing becomes a necessary part of unit testing. In practice, the distinction between unit and integration testing is not clear-cut.

In the introduction to this unit, we mentioned that at any one time we would concentrate on coding just the methods required for a particular use case; that is, we would take a **use case-driven** approach to implementation.

SAQ 5

Why does a use case-driven approach to implementation lead naturally into integration testing?

ANSWER.....

A use case normally involves interaction between objects of more than one core system class; that is, its methods involve more than one core system class.

In a use case-driven approach, having coded and tested the methods involved in one use case, another use case is tackled. Once the methods of this new use case are working it is essential to rerun all previous tests to make sure that everything still works, a process known as **regression testing**. Regression testing is vital not only while the system is being created but when the system is operational, because every time the code is changed – to extend the system, improve it or rectify a fault – bugs may accidentally be introduced that were not there before. JUnit facilitates regression testing, gathering together all the test classes for a package or a group of related packages into a **test suite**, which can be run each time anything is changed.

System testing

When components are large, containing many classes, it can be difficult during integration testing to identify exactly where an error occurs. One way to address this is to code and test each component independently before adding them into the system one by one, as will be done with the Hospital System components identified in the solution to SAQ 4. As each component is added, integration and regression tests are run. Testing the complete system against its specification once all components are integrated is known as **system testing**; this also involves testing aspects of the system such as performance, security, system start up, etc.

This section introduced some key ideas in testing. Testing is a huge area, and far from being something that is clearly defined, it is an active field of research. In M256 we advocate test-driven development because we believe this is a sound approach, both theoretically and in practice. In the next two sections of this unit, though, you will be required to concentrate on writing code and you will temporarily disregard testing, not because it is unimportant but simply because it will be helpful for you initially to concentrate on one aspect at a time.

Later on in this unit you will take a test-driven approach to implementing parts of the Hospital core system, using JUnit to manage your testing. Although testing is complicated, all the examples you will meet are very simple, involving only some of the tests that could be appropriate and hence allowing you to see how testing and coding work together.

3

Beginning implementation

In this section, you will learn about the first stage in implementing a core system – the **first code iteration** as we will call it – in which the basic structure of each class (i.e. its attributes, link references, constructor(s), getter methods for attributes and `toString()` method) is implemented.

3.1 Instance variable declarations

The starting point for implementation is the structural part of the implementation model, which specifies the classes that need to be created, and the attributes and link references that need to be implemented as instance variables.

SAQ 6

Using the implementation model for the Hospital System given in Section 5 of the *Handbook*, list the classes that need to be implemented in the package `hospitalcore`.

ANSWER.....

HospCoord, Ward, Team, Patient, Doctor, JuniorDoctor and ConsultantDoctor.

Consider the class `Patient`. Here is part of its class description in the Hospital System implementation model.

Class	<code>Patient</code>	A patient in the hospital
Attributes		
	<code>private Person person</code>	The person who is a patient
Links		
	<code>private Ward ward</code>	
		References the linked <code>Ward</code> object.
	<code>private Team team</code>	
		References the linked <code>Team</code> object.
	<code>private Collection<Doctor> doctors</code>	
		References a collection of all the linked <code>Doctor</code> objects.
	<code>[ConsultantDoctor consultantDoctor</code>	
		References the linked <code>ConsultantDoctor</code> object.]

Recall that `Person` is defined in the `m256people` package.

Based on this information, the `Patient` class will need the following instance variable declarations.

- To implement attributes:

```
private Person person; /** the person who is a patient */
```

- To implement link references:

```
private Ward ward;      /** the linked Ward object */
private Team team;      /** the linked Team object */
private Collection<Doctor> doctors; /** all the linked Doctor objects */
```

Note that the link reference `consultantDoctor` is not implemented as an instance variable. The brackets [...] on this element of the `Patient` class description record the design decision that the link from a `Patient` object to a `ConsultantDoctor` object, which is a link of the derived association `isResponsibleFor`, will in fact be derived by the system.

You will see in Exercise 17 how this derivation is implemented.

Exercise 1

Refer to the class description for `Ward` in the Hospital System implementation model, in Section 5 of the *Handbook*. Concentrate on the parts headed **Attributes** and **Links**.

- Explain why there should be no instance variable declaration for the attribute `numberOfFreeBeds`.

- Give the instance variable declarations required for the `Ward` class.

Discussion.....

- There should be no instance variable for `numberOfFreeBeds`, because during design it was decided that the system would derive the values of the attribute `numberOfFreeBeds` whenever they are required, rather than implementing the attribute as an instance variable.

The decision that the system should derive the values of the attribute `numberOfFreeBeds` is indicated by the surrounding brackets [...] on this element of the `Ward` class description.

- The instance variable declarations for `Ward` are as follows:

- To implement attributes:

```
private String name;      /** the unique name of the ward */
private Sex type;         /** whether the ward is for male or female
                           * (M or F) patients
                           */
private int capacity;     /** the maximum number of patients that
                           * can be on the ward at any one time
                           */
```

- To implement link references:

```
private Collection<Patient> patients; /** all the linked Patient objects */
```

3.2 Constructors

In the first code iteration, constructors will be implemented – this will enable objects to be created so that tests may be run when the process of implementing use cases begins.

Returning to the `Patient` class, the implementation model specifies the following constructor:

```
Patient(Name aName, Sex aSex, M256Date aDate)
```

Post-condition: initialises a new `Patient` object with the given attribute values and age according to `aDate`.

In implementing the `Patient` constructor, what kinds of decision does the programmer have to make? Well, for some things there is little choice. Since a `Patient` object's people-related features are implemented by its `person` instance variable the constructor must create a `Person` object with the supplied attribute values. This object must be assigned to the newly created `Patient` object's `person` instance variable, as follows.

```
Patient(Name aName, Sex aSex, M256Date aDate)
{
    person = new Person(aName, aSex, aDate);
    ...
}
```

The `Patient` instance variables `ward`, `team` and `doctors`, which implement link references, must also be taken into account. When a `Patient` object is initialised, as can be seen from the specification of the constructor, no information is recorded about any `Ward`, `Team` or `Doctor` objects that should be linked with the `Patient` object; such information is recorded separately, using various `Patient` methods. However, since `doctors` is intended to reference a collection of `Doctor` objects, the `doctors` instance variable can be set in the constructor to reference an empty collection, ready to hold any `Doctor` objects as they are linked later on. The collection chosen must conform to the declaration of the `doctors` instance variable, which was given above as:

```
private Collection<Doctor> doctors;
```

SAQ 7

Suppose that the `doctors` instance variable of a `Patient` object references an object of class `A`. How must `A` be related to the interface `Collection<Doctor>`?

ANSWER.....

The class `A` must implement the interface `Collection<Doctor>`.

When it comes to choosing a class for a particular collection of objects, our rule of thumb will be to choose the simplest collection that fulfils the requirements. The `doctors` instance variable is used to record `hasTreated` links between a `Patient` object and `Doctor` objects. That is, `doctors` will reference a collection of all the `Doctor` objects corresponding to doctors who have treated the patient. There are no special requirements on such a collection: no ordering is required, and neither are duplicate elements to be recorded, for example. Hence the simplest adequate collection which implements `Collection<Doctor>` is `HashSet<Doctor>`.

Exercise 2

Complete the coding of the `Patient` constructor.

Discussion.....

```
Patient(Name aName, Sex aSex, M256Date aDate)
{
    person = new Person(aName, aSex, aDate);
    doctors = new HashSet<Doctor>( );
}
```

No method comment is shown here for this method; the issue of method comments is taken up in Subsection 3.3.

In the Hospital System implementation model, several classes do not have a constructor specified (`Ward`, for example). This is because we have specified this system with a deliberately simple set of use cases, in order not to obscure essential teaching points. For example, there is no use case concerning the establishment of a new ward within the hospital, which would have led to the specification of a `Ward` constructor. Of course, when it is run, the Hospital System will nevertheless require the presence of a `Ward` constructor in order for the hospital to be initially populated with wards. In Exercise 4 you will be asked to implement a constructor for `Ward`, and you will see that constructors are similarly implemented for the other core system classes.

3.3 Getter methods for attributes

The first code iteration will also implement getter methods for the attributes defined in the implementation model. These are the methods which return information about the values of attributes, for example, the `Ward` method `getName()`. (Methods which return information about values of link references, such as `getPatients()` in `Ward`, will be implemented later, as the need for them arises in particular use cases.)

There are several reasons for implementing getter methods for attributes at this stage.

- ▶ The code for a getter method is usually simple, and often automatically generated by an IDE.
- ▶ Getter methods are often required in performing tests of other methods, to verify the state of an object.
- ▶ You may recall that, in *Unit 7*, we decided to specify a getter method for each attribute of a class, whether or not there was a use case that required such a method. This was to allow for the possibility of different user interfaces in the future having different display requirements (for example, displaying a patient's date of birth rather than their age). A purely use case-driven approach to implementation will omit to code those getter methods which are not involved in the present form of the use cases. Establishing all attribute getter methods in the first iteration ensures that even those methods that are not currently required for a use case are implemented.

Coding a getter method according to its specification is mostly very straightforward, as you would expect. Here is the specification of the `Ward` getter method `getName()`.

```
public String getName()
    Post-condition: returns name.
```

And here is code corresponding to this specification.

```
/**
 * Returns the name of this ward.
 *
 * @return name
 */
public String getName()
{
    return name;
}
```

The only point of note about this method is its comment. The description of the method in the specification has been implemented as a doc comment, delineated by `/**` and `*/`. A doc comment on a method typically has two parts: first, a description; then particular items of information about the method (e.g. return value, parameters, etc.), which are prefixed with **block tags**. In this example, the comment includes:

- ▶ an informal description of the method in real-world terms;
- ▶ one block tag, `@return`, after which is placed information about the return value of the method.

Javadoc formats the elements of a doc comment as shown in the example in Figure 2, using its description to provide a general description of the method, and its block tags to highlight particular information – in this case, the method's return value.

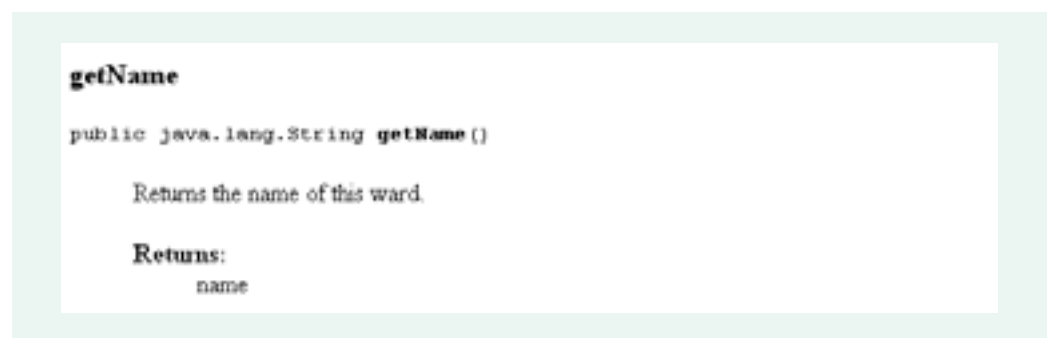


Figure 2 Javadoc documentation for the `getName()` method of `Ward`

Exercise 3

This exercise asks you to write (on paper) code for some getter methods. In each case you should give the entire method, and include doc comments following the pattern of the comments shown above for the method `getName()`.

(a) The following are the specifications of the remaining getter methods for `Ward`.

Protocol

```
public Sex getType()
    Post-condition: returns type.

public int getCapacity()
    Post-condition: returns capacity.

public int getNumberOfFreeBeds()
    Post-condition: returns numberOfFreeBeds.
```

Write code for each of these three methods. For `getNumberOfFreeBeds()` you will need to remember that the attribute `numberOfFreeBeds` is not implemented as an instance variable; instead its values should be derived, using invariant 3 of the Hospital System, which reads as follows:

For each `Ward` object, the value of its `numberOfFreeBeds` attribute is equal to the value of its `capacity` attribute minus the number of `Patient` objects to which it is linked.

- (b) Write code for the `Patient` getter method `getName()` corresponding to the following specification:

```
public Name getName()
```

Post-condition: returns `name`.

In your code you will need to remember that a `Patient` object's person-like qualities are represented by its `person` attribute, and you will need to use a method of the `Person` class, which is specified in the implementation model in Section 5 of the *Handbook*.

Discussion.....

- (a) The code for the getter methods is as follows:

```
/**
 * Returns the type of this ward.
 *
 * @return type
 */
public Sex getType()
{
    return type;
}

/**
 * Returns the capacity of this ward.
 *
 * @return capacity
 */
public int getCapacity()
{
    return capacity;
}

/**
 * Returns the number of free beds in this ward.
 *
 * @return numberOfFreeBeds
 */
public int getNumberOfFreeBeds()
{
    return capacity - patients.size();
}
```

Notice that the comment for `getNumberOfFreeBeds()` mentions `numberOfFreeBeds`. Although `numberOfFreeBeds` is not implemented as an instance variable, it is still considered an attribute of `Ward` by any client invoking this method.

(b) The code for `getName()` is as follows:

```
/**
 * Returns the name of this patient.
 *
 * @return name
 */
public Name getName()
{
    return person.getName();
}
```

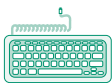
3.4 The first code iteration

Before progressing to examine and complete an initial implementation of the Hospital System, one more aspect of the implementation model that will be implemented in the first code iteration needs to be mentioned: `toString()` methods. Such methods were specified in *Unit 9*, Subsection 3.6, for each class of the core system, in accordance with commonly accepted good practice.

All the elements that will be implemented initially have now been discussed. Each class will be created and its instance variables declared. It will be equipped with a constructor, with getter methods for attributes, and with a `toString()` method, corresponding to the specifications in the implementation model. In Exercise 4 you will be asked to:

- ▶ examine a partially complete first code iteration of the Hospital System;
- ▶ implement the first code iteration of the `Ward` class.

This will also involve consideration of an implementation that we have provided of the utility package `m256people` (for simplicity you will not be involved in the implementation of this package).



Exercise 4

Before beginning this exercise you should have worked through Subsections 3.3 and 3.4 of the *NetBeans Guide* (as noted in Section 1 of this unit). You will not be creating a project from scratch in this exercise, but you will be creating a class and writing source code using the source editor.

In NetBeans open the project `Hospital_Ex_4`, which is in the folder `M256\M256Code\Exercises\Unit10`.

This project contains a partially complete first code iteration of the Hospital System. The package `m256people`, which was specified in *Unit 9*, has already been implemented, as has the initial code for all the core system classes in `hospitalcore` except for `Ward`.

- (a) Examine the package `m256people`.
- (i) What classes and enumerated types are defined in this package?
 - (ii) Briefly examine the code for each class and enumerated type, and confirm that the implementation of this package conforms to its specification in the Hospital System implementation model, in Section 5 of the *Handbook*. (Do not ponder the code of any method for long.) How are the values of the enumerated type `Sex` defined?

- (iii) Generate the Javadoc documentation for this project, and identify where the text associated with the block tags in the doc comment for the `equals()` method of `Name` appears in the Javadoc documentation.

Note: NetBeans will warn you that the `Ward` class has not been implemented (and so the project will not yet compile). The documentation should nevertheless still be generated.

- (b) Next, turn to the package `hospitalcore`. In this package are fledgling definitions of the classes of the core system, except for `Ward`.

- (i) Briefly examine the definition of each class. Describe any elements common to the majority of the classes.
- (ii) Add a new class called `Ward` to the package `hospitalcore`. The class should be defined to implement the interface `java.io.Serializable`. The following import statements are required – why is this?

```
import java.util.*;
import m256people.*;
```

- (iii) Now implement the following in `Ward`:

- ▶ The declaration of instance variables as in Exercise 1(b) (`name`, `type`, `capacity`, `patients`).
- ▶ A constructor, with code as given below.

```
Ward(String aName, Sex aSex, int aCapacity)
{
    name = aName;
    type = aSex;
    capacity = aCapacity;
    patients = new HashSet<Patient>();
}
```

- ▶ Getter methods for attributes, as discussed in Subsection 3.3 (`getName()`, `getType()`, `getCapacity()`, `getNumberOfFreeBeds()`).
- ▶ A `toString()` method, corresponding to the following specification from the implementation model.

```
public String toString()
```

Post-condition: returns a string representation of the receiver's `name`, `type`, `capacity` and linked `Patient` objects.

Ensure that the project compiles (i.e. builds). (It will not run, since there is no `main()` method.)

Discussion.....

- (a) An examination of the package `m256people` reveals the following.

- (i) The package `m256people` defines the classes `Name` and `Person`, and the enumerated types `Sex` and `Grade`.
- (ii) The implementation of this package is straightforward, and it should be clear that each class conforms to its specification. The enumerated type `Sex` is defined as an `enum` and declares two constants `M` and `F`, thereby defining the values of this type.
- (iii) Javadoc should produce the information about the `equals()` method shown in Figure 3. This shows that the text associated with the `@param` and `@return` block tags is incorporated into the documentation under the headings 'Parameters' and 'Returns' respectively in the method description. The 'Overrides' information is generated automatically.

To generate the Javadoc, from the Build menu choose Generate Javadoc for "Hospital_Ex_4", or alternatively right-click on `Hospital_Ex_4` in the Projects window and select Generate Javadoc for Project.

You can enter the required code directly, using the Source Editor, or you can use some of NetBeans' automated facilities, described in Subsection 3.4 of the *NetBeans Guide*.

If a browser window does not open automatically, the Javadoc can be accessed via the Files window in NetBeans (viewable by selecting Windows | Files). In the Files window expand the Hospital_Ex_4 node, next expand the newly revealed dist node, and from there expand the javadoc node. Right-click on the `index.html` file and select View.

```
equals

public boolean equals(java.lang.Object o)

Returns true if o is a Name object with title, firstName and surname equal to those of the
receiver otherwise returns false.

Overrides:
    equals in class java.lang.Object
Parameters:
    o - the object to be checked for equality with the receiver
Returns:
    true if the receiver and argument are equal, false otherwise
```

Figure 3 Javadoc documentation for the `equals` method of `Name`

(b) The answers relating to the classes of the `hospitalcore` package are as follows.

- (i) Here are some fairly obvious things that the classes have in common.
 - ▶ Each class is defined inside the package `hospitalcore`, via the following statement:


```
package hospitalcore;
```
 - ▶ Each class except `Team` imports the package `m256people` via the following statement:


```
import m256people.*;
```
 - ▶ Each class except `ConsultantDoctor` and `JuniorDoctor` implements the interface `java.io.Serializable` by virtue of the code `implements java.io.Serializable`. (`ConsultantDoctor` and `JuniorDoctor` implement this interface since they inherit from `Doctor`, which implements it.)

You may have worked with `java.io.Serializable` before, but here you need only be aware that implementing this interface enables information about the state of a class's objects to be stored in a file, so that objects can be recreated when the system restarts. (You will learn more about this aspect of the Hospital System in *Unit 14*.)
 - ▶ Each class, except `ConsultantDoctor`, has a `toString()` method. (`ConsultantDoctor` inherits this method from `Doctor`.)
 - ▶ Each class also has a constructor.
- (ii) The package `java.util` must be imported in order to access the Java Collections Framework. This is required, for example, to implement the instance variable `patients` using `HashSet`.

The package `m256people` must be imported in order to access the enumerated type `Sex`, the type of the `Ward` instance variable `type`.
- (iii) Examine our implementation of the `Ward` class in the project `Hospital_Ex_4_Sol`.

In this section you have seen how to implement the basic structural elements of a class, based on the specifications in the implementation model of its attributes and link references, its constructor(s), its attribute getter methods and its `toString()` method.

In the next section you will examine a version of the Hospital System in which the methods for a particular use case have been implemented, and you will trace the correspondence between the implementation model and the code.

4

From implementation model to code

In this section, you will be introduced to the use of an implementation model as a blueprint in implementing the methods required for a particular use case.

As already mentioned, the approach to this stage of development will be both use case driven and test driven. This means that the use cases will be implemented one by one. For each method of a use case, test cases will first be specified, corresponding tests will be written, and then the method code will be written and the tests performed. A use case-driven approach is appropriate since for each use case the implementation model includes dynamic models, which show how the methods involved in that use case 'fit' together. Such information about how a method interacts with others is vital in determining how to code it, as you will see.

In this section, we introduce the implementation part of the process – the coding of methods. In Section 5 you will be introduced to the associated testing process.

4.1 Using dynamic models

Recall that, for each use case, the design phase resulted in one or more dynamic models. Each dynamic model, set in the context of a particular scenario of example objects, consists of a walk-through and a sequence diagram, which together capture how the core objects will interact to perform the tasks required by the use case. These dynamic models, along with the individual method specifications in the class descriptions, form the basis for coding methods.

To see how all this works, consider the *Admit Patient* use case. Each use case is initiated by the user interface sending a coordinating message to the coordinating object, which is an object of the class *HospCoord*. Here is the specification of the *Admit Patient* coordinating method from the *HospCoord* class description.

```
public Ward admit(Name aName, Sex aSex, M256Date aDate, Team aTeam)
```

Post-condition: if there is no *Ward* object of the appropriate type with at least one free bed then *null* is returned.

Otherwise a new *Patient* object, *aPatient*, is created with the supplied attribute values, and age according to *aDate*, and:

- (i) *aPatient* is linked to *aWard*, where *aWard* is a *Ward* object of the appropriate type with the greatest number of free beds, and *numberOfFreeBeds* of *aWard* is decremented;
- (ii) *aPatient* is linked to *aTeam*;
- (iii) *aPatient* is linked to the *ConsultantDoctor* object that is linked to *aTeam*;
- (iv) *aWard* is returned.

On receipt of a coordinating message the coordinating object will respond by initiating a sequence of messages within the core system. The methods corresponding to these messages comprise what needs to be implemented for the use case.

To decide what needs to be implemented for *Admit Patient*, the two dynamic models for this use case will be examined. These are based on what were referred to as the *typical* and *alternative scenarios*. Recall that a scenario involves an invented set of objects with

particular states and links with each other. The dynamic models for the two scenarios for *Admit Patient* are given in the *Dynamic models* part of Section 5 of the *Handbook*, and you should refer to these now. In the typical scenario there is a suitable ward for the patient; in the alternative scenario there is not.

First, we will consider the typical scenario, where there is a suitable ward. The object diagrams (Figures 4 and 5 in the *Handbook*) illustrate the objects and links involved in the typical scenario. The walk-through describes the steps that should be taken in the core system for this scenario; the sequence diagram (Figure 6 in the *Handbook*) shows the sequence of messages that should pass through the system to achieve the steps of the walk-through. The walk-through and sequence diagram express what should happen for this particular scenario, but the steps involved are applicable to any similar scenario for this use case and so the designs expressed are general.

When implementing a use case, the sequence diagrams define what methods need to be implemented and how the methods are related (which methods invoke which other methods). Then, for each method, its method specification defines which steps of the walk-through the method code should achieve.

In this case the sequence diagram for the use case starts with a coordinating message `admit(Ms Lynda Snell, F, 29/05/47, team2)` sent from the user interface to the coordinating object `hospital`. Looking at the method specification you can see that the method should, in summary, be coded to achieve the following:

- ▶ find a `Ward` object of the correct type with the greatest number of free beds to take this `Patient` object (if there is no such `Ward` object then the alternative scenario is appealed to: we turn to this shortly);
- ▶ create a new `Patient` object to represent the patient;
- ▶ create the links between the new `Patient` object, the `Ward` object and the `Team` object given in the coordinating message from the user interface;
- ▶ return the `Ward` object.

SAQ 8

Why should there be no code to create a link between the `Patient` object and a `ConsultantDoctor` object, corresponding to part (iii) of the specification for the `admit(aName, aSex, aDate, aTeam)` method?

ANSWER.....

At the design stage it was decided that such links, which are links of the derived association `isResponsibleFor`, would not be explicitly implemented in code, but instead would be derived. This is reflected in the walk-through steps, but not the method specification (since the method should appear, to a client, to set the link). This is a similar situation to that of a derived attribute (see Exercise 3(a)), which to a client is just as 'real' as any other attribute.

The sequence diagram shows certain messages being initiated from within `hospital`'s activation rectangle, which represents `hospital`'s processing of the message `admit(Ms Lynda Snell, F, 29/05/47, team2)`. So these messages, which are listed below, will need to be sent by `hospital` from within its `admit(aName, aSex, aDate, aTeam)` method.

- ▶ `getType()` messages to `Ward` objects.
- ▶ `getNumberOfFreeBeds()` messages to `Ward` objects.
- ▶ An `admit(aWard, aTeam)` message to a `Patient` object.
- ▶ `hospital` also invokes the `Patient` constructor.

The implementation of the method `admit(aName, aSex, aDate, aTeam)` therefore requires the presence in the relevant classes of methods corresponding to these messages. Figure 4 shows these messages emboldened on the sequence diagram from Figure 6 in the *Handbook*.

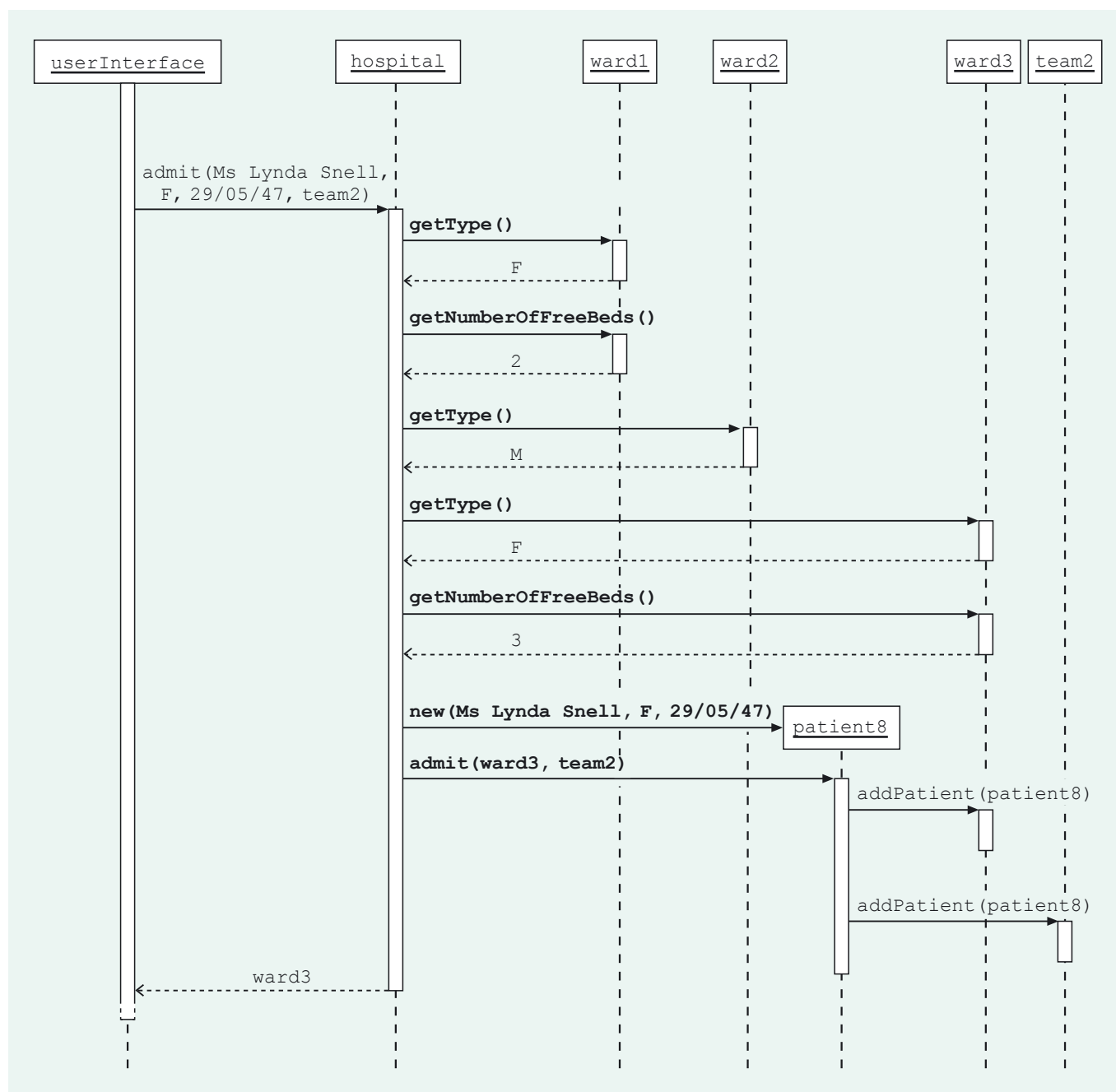


Figure 4 Messages initiated from within hospital's activation rectangle

Exercise 5

The sequence diagram in Figure 4 shows two messages that `hospital` does not send: `addPatient(patient8)` to a `Ward` object; and `addPatient(patient8)` to a `Team` object.

- Explain how the sequence diagram indicates which method these two messages will be sent from.
- Will any messages be sent to other objects from within either the `addPatient(aPatient)` method in `Ward` or the `addPatient(aPatient)` method in `Team`?

Discussion.....

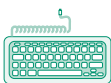
- (a) The sequence diagram shows `patient8` receiving the message `admit(ward3, team2)`. This causes its method `admit(aWard, aTeam)` to be invoked: the processing of this method is represented by `patient8`'s activation rectangle. Since both the `addPatient(patient8)` messages emanate from this activation rectangle, they must be sent from within the `Patient` method `admit(aWard, aTeam)`.
- (b) The sequence diagram shows no further messages being sent in either case: there are no messages to other objects emanating from the activation rectangle for either `ward3` or `team2`. This means that a `Ward` object, for example, calls on no other objects to help in the execution of its `addPatient(aPatient)` method. It is possible that the programmer might decide to implement internal helper methods in `Ward`, for a `Ward` object to use in executing its `addPatient(aPatient)` method, but we will not pursue this further.

Exercise 5, and the preceding discussion, indicates the significance of the order in which methods are implemented. It is sensible to implement first those methods (if there are any) which do not rely on other methods, then those which rely only on methods that are already implemented, and so on. We will return to this idea briefly in Section 6; for present purposes it suffices to note that the following would be a sensible order of implementation for the methods of *Admit Patient*:

- 1 `getType()` in `Ward` (a getter method whose method was already implemented in the first code iteration);
- 2 `getNumberOfFreeBeds()` in `Ward` (as above; already implemented);
- 3 The `Patient` constructor (already implemented in the first code iteration);
- 4 `addPatient(aPatient)` in `Ward`;
- 5 `addPatient(aPatient)` in `Team`;
- 6 `admit(aWard, aTeam)` in `Patient`;
- 7 `admit(aName, aSex, aDate, aTeam)` in `HospCoord`.

4.2 Examining method code

In Exercises 6 and 7 you will examine the implementation of the methods listed above, comparing each with its specification in the relevant class description, and tracing the correspondence of the methods with the sequence of messages in the sequence diagram. Remember that as a programmer you would be writing tests before coding methods, but to make things easier for you to follow we are temporarily ignoring testing and concentrating only on the coding.



Exercise 6

In NetBeans open the project `Hospital_Ex_6`, which is in the folder `M256\M256Code\Exercises\Unit10`. (Note that this project will also be used for Exercise 7).

This project builds on `Hospital_Ex_4_Sol`. Not only is the first code iteration complete (each class is defined, with instance variables declared, and attribute getter methods, constructors and `toString()` methods implemented), but all the methods specified for the *Admit Patient* use case have been implemented.

- (a) Examine the implementation of the method `addPatient(aPatient)` in `Ward`.
- (i) Explain how the code for this method implements the method specification in the `Ward` class description, which is as follows:

```
void addPatient(Patient aPatient)
```

Post-condition: a reference to `aPatient` is recorded; `numberOfFreeBeds` is decremented.

- (ii) Neither the method `addPatient(aPatient)` in `Ward` nor the method `addPatient(aPatient)` in `Team` appears in the documentation generated by Javadoc. Check this and explain why it is so.
- (b) Here is the specification of the method `admit(aWard, aTeam)` in the `Patient` class description.

```
void admit(Ward aWard, Team aTeam)
```

Post-condition: a reference to `aWard` is recorded, `aWard` records a reference to the receiver, and `numberOfFreeBeds` of `aWard` is decremented. A reference to `aTeam` is recorded; and `aTeam` records a reference to the receiver. A reference to the `ConsultantDoctor` object linked to `aTeam` is recorded.

Compare this specification with the sequence diagram in Figure 4. On receiving a message of the form `admit(aWard, aTeam)` a `Patient` object sends two messages as follows.

- 1 A message of the form `addPatient(aPatient)` is sent to the `Ward` object referenced by `aWard`. This achieves the part of the specification for `admit(aWard, aTeam)` which states that 'a reference to the receiver, and `numberOfFreeBeds` of `aWard` is decremented', by delegating responsibility for this task to the `Ward` object.
 - 2 A message of the form `addPatient(aPatient)` is sent to the `Team` object referenced by `aTeam`. This implements the part of the specification which states that 'a reference to the receiver'.
- (i) Why are there no messages in the sequence diagram relating to the parts of the specification which state that 'a reference to `aWard` is recorded' and that 'a reference to `aTeam` is recorded'?
- (ii) Examine the code of the method `admit(aWard, aTeam)` in `Patient`. Explain how the code implements the method specification above.

Discussion.....

- (a) Regarding the `addPatient(aPatient)` method in `Ward`:

- (i) The method is coded as follows.

```
void addPatient(Patient aPatient)
{
    patients.add(aPatient);
}
```

The instance variable `patients` of a `Ward` object implements this object's link reference to the collection of `Patient` objects corresponding to the patients who are on the ward. The reference to `aPatient` is therefore recorded via the `patients.add(aPatient);` statement.

The method does not explicitly decrement `numberOfFreeBeds`, since `numberOfFreeBeds` is derived, and is not implemented as an instance variable.

Note, however, that the method `addPatient(aPatient)` has the *effect* of decrementing `numberOfFreeBeds`, as perceived by a client using the getter method `getNumberOfFreeBeds()`. The return value of `getNumberOfFreeBeds()` is determined by the calculation `capacity - patients.size()`. So when an

The marginal note alongside Exercise 4 reminds you how to generate Javadoc.

Access modifiers were discussed in *Units 5 and 7*.

invocation of the `addPatient(aPatient)` method increases the size of `patients` by 1, `getNumberOfFreeBeds()` returns a value 1 less than previously.

- (ii) Neither of the methods `addPatient(aPatient)` in `Ward` or in `Team` appears in the Javadoc documentation, because they both have package accessibility (i.e. no access modifier is defined for either), and Javadoc does not produce documentation for methods with package (or private) accessibility. (Nevertheless, comments of some form should always be provided on methods with package, and private, accessibility, for the benefit of those developing and maintaining the package.)

Notice that, for each class, its methods have been classified in the code under one of the headings *public protocol* and *package protocol*.

- (b) Regarding the `admit(aWard, aTeam)` method in `Patient`:

- (i) There are no messages relating to these parts of the specification since these parts simply require the `Patient` object to set its own references. No messages are needed for this.
- (ii) The method is coded as follows.

```
void admit(Ward aWard, Team aTeam)
{
    ward = aWard;
    ward.addPatient(this);
    team = aTeam;
    team.addPatient(this);
}
```

The code corresponds with the specification as follows.

- ▶ The statement `ward = aWard;` implements the recording of a reference to `aWard`.
- ▶ The statement `ward.addPatient(this);` sends `aWard` the message `addPatient(this)`. This implements the recording of a reference by `aWard` to the receiver `Patient` object (and decrementing `numberOfFreeBeds` of `aWard`).
- ▶ The statement `team = aTeam;` implements the recording of a reference to `aTeam`.
- ▶ The statement `team.addPatient(this);` sends `aTeam` the message `addPatient(this)`. This implements the recording of a reference by `aTeam` to the receiver.
- ▶ There is no code implementing the recording of a reference to the `ConsultantDoctor` object linked to `aTeam`, since this is a link of the derived association `isResponsibleFor`.

In Exercise 7 you will complete your review of the implementation of the *Admit Patient* use case by examining the implementation of the `HospCoord` coordinating method `admit(aName, aSex, aDate, aTeam)`. To prepare for this, you need to consider the second dynamic model for this use case. The context for this second model is the alternative scenario, in which there is no suitable ward for the patient.

Refer now to the walk-through and sequence diagram (Figure 8) for this alternative scenario for *Admit Patient* in the *Dynamic models* part of Section 5 of the *Handbook*. You can see that the response of the core system when there is no suitable ward is – having determined this fact – to return `null`.

Exercise 7



Return to NetBeans and the project `Hospital_Ex_6`. Examine the method `admit(aName, aSex, aDate, aTeam)` in the class `HospCoord`.

- (a) Which part of the code corresponds to the search for a suitable ward?
- (b) Describe in outline how the code in part (a) works, stating which services of other objects are required. Note that `wards` is an instance variable of `HospCoord`, referencing a collection of all `Ward` objects.
- (c) Which code determines whether a suitable `Ward` object has been found?
- (d) What does the code do if no suitable `Ward` object has been found? Which sequence diagram does this correspond to?
- (e) If a suitable `Ward` object has been found:
 - (i) what statements are executed?
 - (ii) to which part of the method specification for `admit(aName, aSex, aDate, aTeam)` do the statements in (i) correspond?
 - (iii) to which part of the sequence diagram in Figure 6 of the *Handbook* do these statements relate?

Discussion.....

- (a) The following code corresponds to the search for a suitable ward.

```
Ward theWard = null;
int mostBeds = 0;
int freeBeds = 0;
for (Ward aWard : wards)
{
    if (aWard.getType() == aSex)
    {
        freeBeds = aWard.getNumberOfFreeBeds();
        if (freeBeds > mostBeds)
        {
            mostBeds = freeBeds;
            theWard = aWard;
        }
    }
}
```

- (b) The code iterates through the collection of `Ward` objects referenced by `wards` to find the `Ward` object (if one exists) of the correct type with the greatest number of free beds. It calls on the services of the `Ward` objects in doing this, by sending them the messages `getType()` and `getNumberOfFreeBeds()`, corresponding to the messages shown on the sequence diagram in Figures 6 and 8 of the *Handbook*.
- (c) The code `if (theWard != null)` determines whether a suitable `Ward` object has been found.
- (d) If no suitable `Ward` object has been found, the code returns `null`, corresponding to the sequence diagram for the alternative scenario in Figure 8 in the *Handbook*.
- (e) If a suitable `Ward` object has been found:
 - (i) The following statements are executed.

```
Patient thePatient = new Patient(aName, aSex, aDate);
thePatient.admit(theWard, aTeam);
```

- (ii) The statements in (i) above correspond with the following part of the method specification:

Post-condition: ... a new `Patient` object, `aPatient`, is created with the supplied attribute values, and age according to `aDate`, and:

- (i) `aPatient` is linked to `aWard`, where `aWard` is a `Ward` object of the appropriate type with the greatest number of free beds, and `numberOfFreeBeds` of `aWard` is decremented;
- (ii) `aPatient` is linked to `aTeam`;
- (iii) `aPatient` is linked to the `ConsultantDoctor` object that is linked to `aTeam`.

The statements create a new `Patient` object, `thePatient`, and then send `thePatient` a message `admit(theWard, theTeam)`. In turn, on receipt of this message, `thePatient` sends `addPatient(thePatient)` messages to `theWard` and to `theTeam`.

- (iii) The statements in (i) above relate to the part of the sequence diagram shown in Figure 5.

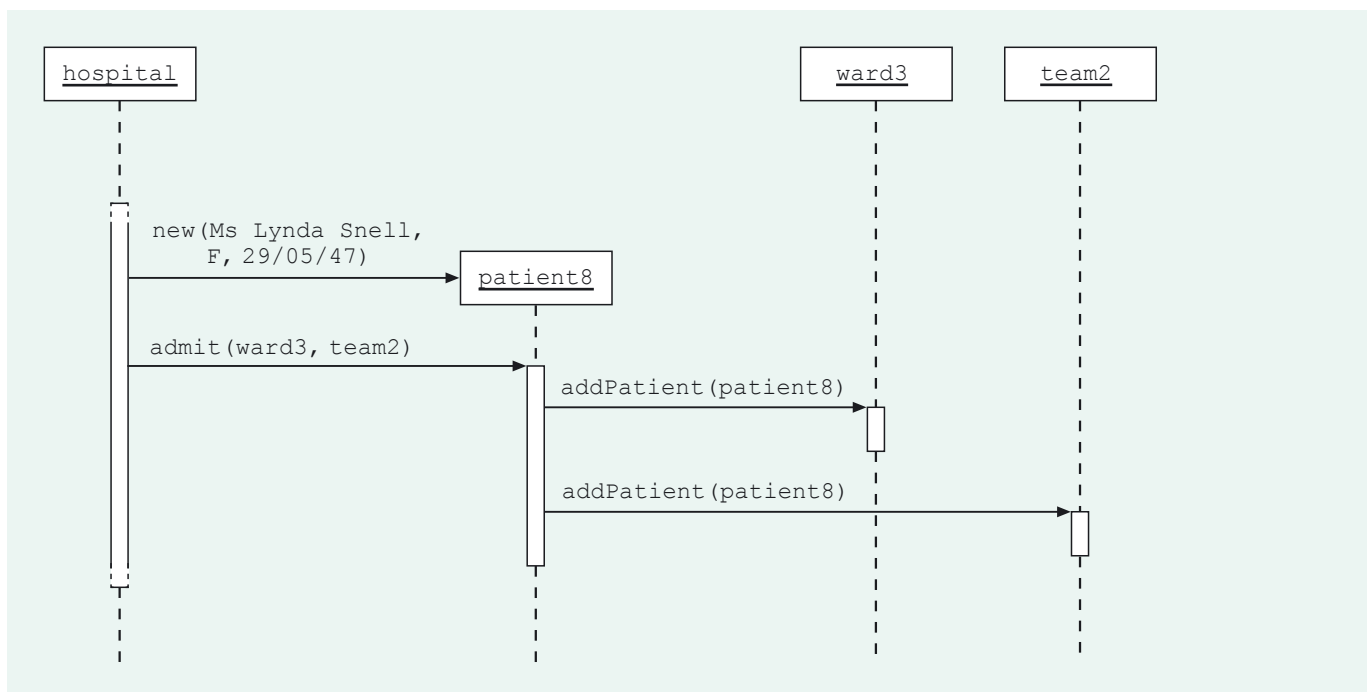


Figure 5 An excerpt from the sequence diagram for the *Admit Patient* coordinating method (a suitable ward exists)

In this section you have examined an implementation of the *Admit Patient* use case. For each method involved, you have confirmed that its code achieves the specification of the method, and you have seen that the messages sent from within the code correspond to those depicted on the sequence diagram.

In the next section you will be introduced to what is involved in specifying test cases, and in writing and running tests for methods. Then in Section 6 you will bring together what you have learnt, in implementing another use case.

5

Testing a method

As you progress to implementing for yourself the methods required for a particular use case, you will take a test-driven approach, by writing tests before you write each method. Here we prepare you for this, by examining the tests appropriate for one particular method: both how the test cases are specified, and how the tests are coded and run. The example we will use is the `Team` method `getPatientsAndWards()`.

5.1 Specifying test cases

SAQ 9

- (a) What is required to specify a test case?
 (b) Describe how a simple set of test cases can be determined from a method specification.

ANSWER.....

- (a) Section 2 stated that the following should be specified:
- ▶ the state of the relevant parts of the system before the test (the fixture), along with any input data;
 - ▶ the expected state of the relevant parts of the system after the test, along with any expected output data.
- (b) Section 2 indicated that test cases should be specified:
- ▶ for each specified aspect of the behaviour of the method;
 - ▶ for each different class of object involved that may be treated differently by the method;
 - ▶ for any different states of objects involved that may be treated differently by the method;
 - ▶ around each boundary.

The specification for the method `getPatientsAndWards()` in the class `Team` is as follows.

```
Map<Patient, Ward> getPatientsAndWards()
```

Post-condition: returns a map of pairs of `Patient` and `Ward` objects, such that for each `Patient` object, `aPatient`, linked to the receiver, the map contains the key-value pair (`aPatient`, `aWard`), where `aWard` is linked to `aPatient`.

SAQ 10

- (a) What distinct behaviours are specified for this method?
 (b) What boundary should be considered?

ANSWER.....

- (a) There is only one specified behaviour, which is returning a `Map` object as described in the post-condition. There are no variations on this: the same thing should happen whatever the objects involved.
- (b) The boundary to be considered is the case where the `Team` object involved (the receiver of a `getPatientsAndWards()` message) is linked to no `Patient` objects.
-

For this method, to test around the boundary we will specify test cases corresponding to:

- ▶ an empty team;
- ▶ a team caring for one patient.

Though there is only a single specified behaviour at stake, there are particular states of the system which the method could be considered likely to handle differently. Given that a `Map` object is involved, and that associating values with keys in a map can easily go wrong, we will also consider:

- ▶ the case where the team cares for several patients, some of whom are in the same ward as each other;
- ▶ the case where the team cares for several patients, no two of whom are in the same ward.

For each of the above four cases in a real project, there may be several different test cases specified: testing, for example, with patients of both sexes, with different numbers of treatments recorded, etc., to check that these variations make no difference to the result, and to increase confidence that the method works correctly. We will however content ourselves with one test in each case.

Having broadly described the test cases, they must now be specified. A useful way to do this is in a structured format such as the one in Table 1. (This table omits the test case for one test, which you are asked to provide in Exercise 8.)

Table 1 Specifying test cases for Team's method `getPatientsAndWards()`

Class Team – Test 1 Test for an empty team	
Method <code>getPatientsAndWards()</code>	Receiver and message <code>team1.getPatientsAndWards()</code>
Fixture and input A Team object <code>team1</code> with patients empty.	Expected result An empty instance of <code>Map<Patient, Ward></code> is returned.
Class Team – Test 2 Test for a team caring for one patient	
Method <code>getPatientsAndWards()</code>	Receiver and message <code>team1.getPatientsAndWards()</code>
Fixture and input A Team object <code>team1</code> with patients containing <code>patient1</code> . A Ward object <code>ward1</code> with patients containing <code>patient1</code> . A Patient object <code>patient1</code> with <code>team = team1</code> and <code>ward = ward1</code> .	Expected result An instance of <code>Map<Patient, Ward></code> is returned, containing precisely the key-value pair <code>(patient1, ward1)</code> .
Class Team – Test 3 Test for a team caring for several patients, some of whom are in the same ward as each other	
Method <code>getPatientsAndWards()</code>	Receiver and message <code>team1.getPatientsAndWards()</code>
Fixture and input A Team object <code>team1</code> with patients containing <code>patient1</code> , <code>patient2</code> and <code>patient3</code> . A Ward object <code>ward1</code> with patients containing <code>patient1</code> and <code>patient2</code> . A Ward object <code>ward2</code> with patients containing <code>patient3</code> . A Patient object <code>patient1</code> with <code>team = team1</code> and <code>ward = ward1</code> . A Patient object <code>patient2</code> with <code>team = team1</code> and <code>ward = ward1</code> . A Patient object <code>patient3</code> with <code>team = team1</code> and <code>ward = ward2</code> .	Expected result An instance of <code>Map<Patient, Ward></code> is returned, containing precisely the following key-value pairs: <code>(patient1, ward1);</code> <code>(patient2, ward1);</code> <code>(patient3, ward2).</code>
Class Team – Test 4 Test for a team caring for several patients, no two of whom are in the same ward	
Method <code>getPatientsAndWards()</code>	Receiver and message <code>team1.getPatientsAndWards()</code>

`Map<Patient, Ward>` is an interface. By 'an instance of `Map<Patient, Ward>`' we mean 'an instance of a class that implements `Map<Patient, Ward>`'.

Exercise 8

Complete Table 1 by specifying a test case for Test 4.

Discussion.....

This final test case is specified in Table 2.

Table 2 Specifying a final test case for Team's method `getPatientsAndWards()`

Class <code>Team</code> – Test 4 Test for a team caring for several patients, no two of whom are in the same ward	
Method <code>getPatientsAndWards()</code>	Receiver and message <code>team1.getPatientsAndWards()</code>
Fixture and input A <code>Team</code> object <code>team1</code> with patients containing <code>patient1</code> , <code>patient2</code> and <code>patient3</code> . A <code>Ward</code> object <code>ward1</code> with patients containing <code>patient1</code> . A <code>Ward</code> object <code>ward2</code> with patients containing <code>patient2</code> . A <code>Ward</code> object <code>ward3</code> with patients containing <code>patient3</code> . A <code>Patient</code> object <code>patient1</code> with <code>team = team1</code> and <code>ward = ward1</code> . A <code>Patient</code> object <code>patient2</code> with <code>team = team1</code> and <code>ward = ward2</code> . A <code>Patient</code> object <code>patient3</code> with <code>team = team1</code> and <code>ward = ward3</code> .	Expected result An instance of <code>Map<Patient, Ward></code> is returned, containing precisely the following key–value pairs: <code>(patient1, ward1);</code> <code>(patient2, ward2);</code> <code>(patient3, ward3).</code>

5.2 Writing and running tests

You will now see how each of the test cases above can be implemented and the corresponding test run using JUnit. In the following discussion, it is assumed that you have worked through Section 6 of the *NetBeans Guide* and are familiar with how JUnit handles testing.

Recall that, in using JUnit the following apply.

- ▶ Each test of a method should take place inside a single test method.
- ▶ For each class, all the test methods for its methods are contained in a corresponding test class.
- ▶ In a test method, the fixture and/or input data is established and the test message is sent. This is then followed by checks that the results are as expected.
- ▶ JUnit's **test runner** automatically executes the test methods and reports the results. The test runner can be executed on a single test class, or on the entire set of test classes for a project. You will only be required to do the latter.
- ▶ Different test methods cannot communicate or interfere with each other.

Here is our code for the test method for Test 1, implementing the test case specified in Table 1.

```
public void test1GetPatientsAndWards()
{
    //set up the fixture
    Collection<Doctor> doctors = new HashSet<Doctor>();
    JuniorDoctor juniorDoctor1 = new JuniorDoctor((new Name("Mr",
        "Jim", "MacDonald")), Grade.ONE);
    ConsultantDoctor consultantDoctor1 = new ConsultantDoctor(new Name("Ms",
        "Betty", "Turpin"));

    doctors.add(juniorDoctor1);
    doctors.add(consultantDoctor1);
    Team team1 = new Team("A&E", doctors, consultantDoctor1);

    //send the test message
    Map<Patient, Ward> results = team1.getPatientsAndWards();

    //check the result
    assertTrue("The map should be empty", results.isEmpty());
}
```

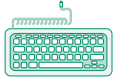
Note the following points about this test method.

- ▶ The final statement in the test method code is an **assertion**, that is, a JUnit test statement which checks that the expected result is the same as the actual result. If the expression `results.isEmpty()` does not return `true`, this assertion will fail, and the failure will be reported by the test runner. Thus the purpose of the assertion is to test whether the `Map` object returned is empty (as described in the message string, which in the case of failure will appear in the test report).
- ▶ For the Hospital System to be in a valid state, a `Team` object must contain both a `JuniorDoctor` object with grade value 1 and a `ConsultantDoctor` object; and it must be headed by that `ConsultantDoctor` object. This is because there are invariants on the system reflecting the real-world constraints that a team must contain both a junior doctor at grade 1 and a consultant doctor who heads the team. In the Hospital System the method `getPatientsAndWards()` will never be invoked when the system is in an invalid state, and we have restricted our tests to such valid situations.
- ▶ The above test method has made no use of JUnit's `setUp()` method. In this method, you can include code for common parts of the fixtures and input data for the test methods. For example, the `Team` object, `team1`, common to all the tests in this case, *could* be created in the `setUp()` method and thereby made available to each of the test methods so that they do not have to individually create this object. However, there are various complications with this approach, and we have chosen not to use `setUp()`.
- ▶ Note that a value of an enumerated type has to be prefixed with the type name: for example, `Grade.ONE` in the above code. This is so that the values of the enumerated type can be distinguished from other identifiers.

We will now move on to the remaining tests. The fixtures for Tests 2, 3 and 4 require that there be `Patient` objects and `Ward` objects, suitably linked to each other and to a `Team` object. To achieve all this, the `Patient` method `admit(aWard, aTeam)`, implemented as part of the *Admit Patient* use case, will be used. There are other ways that the fixture could be established, but they all involve methods of other use cases, because a test method cannot directly set the instance variables of the objects involved in the test. Test methods can become quite complicated, requiring objects to be created and linked,

and depending on other methods as well as constructors. Testing a method in complete isolation from other methods is usually impossible for all but the simplest cases.

You will now look at how the remaining tests can be coded, and at what is involved in running them using JUnit.



Exercise 9

Before beginning this exercise, you should have worked through Section 6 of the *NetBeans Guide* (as noted in Section 1 of this unit).

In NetBeans open the project `Hospital_Ex_9`, which is in the folder `M256\M256Code\Exercises\Unit10`.

This is the same implementation of the Hospital System as in `Hospital_Ex_6`, but with three differences, as follows.

- 1 Within the package `hospitalcore` in Test Packages is a test class `TeamTest`, which contains test methods for all the `getPatientsAndWards()` tests. The test method for Test 4 has been commented out.
- 2 The method `getWard()` of `Patient` has been implemented. This is required for the correct implementation of the `Team` method, `getPatientsAndWards()`, that you will be testing.
- 3 The method you will be testing, `getPatientsAndWards()` of `Team`, has been coded to produce a rather strange result, which will be revealed in this exercise.

- (a) Examine the test class `TeamTest` and then execute the test runner on the project, following the instructions in Section 6 of the *NetBeans Guide*. Note the result.
- (b) Uncomment the test method for Test 4 (remove the `/*` and `*/` surrounding the method) and execute the test runner again. You should now find that the test fails! This is due to the rather bizarre coding of the `getPatientsAndWards()` method of `Team`, which was deliberately designed to give this example of failure.
- (c) The failure noted in part (b) is due to the insertion of the following statement, contrived specifically to make Test 4 fail.

```
if ((new HashSet<Ward>(results.values())).size() == 3)
{
    results.clear();
}
```

Correct the method code by removing this statement from the `Team` method `getPatientsAndWards()`, and execute the test runner again.

Discussion.....

- (a) It should be reported that all three tests were passed.
- (b) Executing the test runner now should result in a report that there was a failure, and you should be pointed to the following line of the Test 4 test method `test4GetPatientsAndWards()`.

```
assertEquals("The right patients and wards should be returned",
    expectedResults, results);
```

- (c) Having corrected the `getPatientsAndWards()` method code, executing the test runner should now result in a report that all four tests were passed.

Tests 2, 3 and 4 are defined as throwing a `ParseException` because `M256Date` throws a `ParseException` if the date supplied in its constructor is not valid. If there is a problem and the exception is thrown, JUnit will report this fact.

In this section you saw how to use a method's specification to specify a basic set of suitable test cases. You learnt how to use a test case specification to write a test method, and you practised running tests using JUnit's test runner.

In the next section you will learn about planning the implementation of a use case, and you will code and test the methods required for the *Treat Patient* use case.

6

Implementing a use case

In previous sections you have:

- ▶ examined code written by the course team for the methods involved in a particular use case, tracing its correspondence with dynamic models and method specifications;
- ▶ learnt about specifying, writing and running tests of a method.

In this section you will bring together this earlier work as we guide you through implementing and testing the methods of a particular Hospital System use case: *Treat Patient*.

6.1 Planning the implementation

Here is the specification of the coordinating method for the *Treat Patient* use case.

```
public void recordTreatment(Patient aPatient, Doctor aDoctor)
```

Pre-condition: aDoctor and aPatient must be linked to the same Team object; if not, an exception is thrown.

Post-condition: aDoctor is linked to aPatient.

This method is invoked when a coordinating message is sent to the coordinating object, hospital, initiating a sequence of messages within the core system. For each such message, the corresponding method must be implemented.

Refer now to the dynamic models for the *Treat Patient* use case in the *Dynamic models* part of Section 5 of the *Handbook*. For this use case, there are two dynamic models. One is based on a *valid scenario*, in which the Doctor object and the Patient object are linked to the same Team object. The other is based on an *invalid scenario*, in which the Doctor object and the Patient object are linked to different Team objects.

Recall that in implementing a use case, the sequence diagrams of the dynamic models give information about what methods need to be implemented for the use case, and how the methods are related (i.e. which methods invoke which other methods). The method specifications give information about what the method code should achieve.

Exercise 10

Look at the sequence diagrams (Figures 10 and 12 in the *Handbook*) for the *Treat Patients* use case.

- (a) Which method or methods are invoked from within the HospCoord method `recordTreatment(aPatient, aDoctor)`?
- (b) Which method or methods are invoked from within the Patient method `recordTreatmentBy(aDoctor)`?
- (c) Which method or methods are invoked from within the Team method `contains(aDoctor)`?

Discussion.....

- (a) The `Patient` method `recordTreatmentBy(aDoctor)` is invoked. You can tell that this is so from the sequence diagrams, in which a message corresponding to this method is sent to a `Patient` object, from within `hospital`'s activation rectangle, which represents its processing of a message of the form `recordTreatment(aPatient, aDoctor)`.
- (b) The `Team` method `contains(aDoctor)` is invoked. Again, the sequence diagrams show that this is the case, because they depict a message corresponding to this method being sent to a `Team` object from within the `Patient` object's activation rectangle, which represents its processing of a message of the form `recordTreatmentBy(aDoctor)`.
- (c) No methods are invoked from within the `Team` method `contains(aDoctor)`. The sequence diagrams show no further messages being sent from within the `Team` object's activation rectangle, which represents its processing of a `contains(aDoctor)` message.

From Exercise 10 you can see that three methods need to be implemented for this use case:

- 1 `recordTreatment(aPatient, aDoctor)` in `HospCoord`;
- 2 `recordTreatmentBy(aDoctor)` in `Patient`;
- 3 `contains(aDoctor)` in `Team`.

Further, you should now be able to identify a sensible order for their implementation, in which you first implement those methods (if there are any) that do not rely on any other methods, then implement those methods that rely only on other methods that are already implemented, and so on. In our simple system, it will always be possible to order the methods in this cumulative fashion; in more complex systems, it may not.

SAQ 11

In what order should the three methods for *Treat Patient* be implemented?

ANSWER.....

They should be implemented in the following order:

- 1 `contains(aDoctor)` (does not rely on any other methods);
- 2 `recordTreatmentBy(aDoctor)` (relies on `contains(aDoctor)`);
- 3 `recordTreatment(aPatient, aDoctor)` (relies on `recordTreatmentBy(aDoctor)`).

Stubs

In more complicated systems the interdependence between methods may mean that it is not possible to implement each method after implementing all the others on which it depends. One way to deal with this is to write stubs for methods that have not yet been implemented. A stub is a 'dummy' method: it does not have the full functionality required of the actual method but it will conform to enough of the method specification to enable it to be used for testing purposes. For example, a stub for a particular method might in a very simple way return a value of the correct type, or it might change an instance variable in a predefined manner but not address any other part of the specification.

Stubs are written into the part of the system being developed and are eventually replaced by the fully working methods. For any substantial system, writing stubs takes time (and in theory, stubs also need to be tested!), so developers usually aim to select an order of implementation which minimises the number of stubs required.

Having identified a suitable order of implementation, following a test-driven approach, what has to be done for each method can be broken down as follows.

- 1 Specify test cases.
- 2 Write code for the tests, which implement the test cases.
- 3 Write the method code.
- 4 Run the tests.
- 5 If any of the tests fail, fix the bug(s) and repeat from Step 3.

When a method has been coded, not only will its own tests be run, but so too will tests for previously implemented methods.

SAQ 12

- (a) What is the name given to the process of rerunning previous tests?
- (b) Why is this process carried out?

ANSWER.....

- (a) The process of rerunning previous tests is called regression testing.
- (b) This process is carried out to check that a change to the system has not caused a problem in part of the system that was previously working.

You learnt about regression testing at the end of Section 2.

6.2 Coding and testing the `contains(aDoctor)` method

Here is the specification of the `Team` method `contains(aDoctor)`.

```
boolean contains(Doctor aDoctor)
```

Post-condition: returns `true` if `aDoctor` is linked to the receiver, `false` otherwise.

Exercise 11

Outline (do not specify) appropriate test cases for the `contains(aDoctor)` method based on its specification.

Discussion.....

There are two distinct behaviours specified, corresponding to the following two cases:

- ▶ the `Doctor` object is linked to the receiver `Team` object;
- ▶ the `Doctor` object is not linked to the receiver `Team` object.

Each of these situations needs to be tested for. In each, there are different classes of object that might affect the behaviour, requiring that tests should be performed where:

- ▶ the `Doctor` object is a `JuniorDoctor` object;
- ▶ the `Doctor` object is a `ConsultantDoctor` object.

As with the tests carried out in Section 5, we will restrict testing to situations that are valid for the Hospital System, so that the `Team` object will contain a `JuniorDoctor` object with grade value 1 and a `ConsultantDoctor` object who heads that `Team` object. In the Hospital System, the method `contains(aDoctor)` will not be invoked unless this is the case.

In this case, then, the collection of `Doctor` objects linked to a `Team` object cannot be empty. Four test cases will therefore be specified.

Table 3 begins the process of specifying the test cases.

Table 3 Beginning to specify tests for `Team`'s method `contains(aDoctor)`

Class <code>Team</code> – Test 1	
Test for a junior doctor who is in the team	
<i>Method</i> <code>contains(aDoctor)</code>	<i>Receiver and message</i> <code>team1.contains(juniorDoctor1)</code>
<i>Fixture and input</i> A <code>JuniorDoctor</code> object, <code>juniorDoctor1</code> , with <code>grade = Grade.ONE</code> . A <code>ConsultantDoctor</code> object, <code>consultantDoctor1</code> . A <code>Team</code> object <code>team1</code> with doctors containing <code>juniorDoctor1</code> and <code>consultantDoctor1</code> .	<i>Expected result</i> <code>true</code> is returned.

Exercise 12

Complete the table by specifying the remaining three test cases.

Discussion.....

Table 4 gives appropriate test case specifications for the remaining three test cases.

Table 4 Specifying tests for Team's method contains(aDoctor)

Class Team – Test 2 Test for a junior doctor who is not in the team	
<i>Method</i> contains(aDoctor)	<i>Receiver and message</i> team1.contains(juniorDoctor2)
<i>Fixture and input</i> A JuniorDoctor object, juniorDoctor1, with grade = Grade.ONE. A ConsultantDoctor object, consultantDoctor1. A JuniorDoctor object, juniorDoctor2. A Team object team1 with: doctors containing juniorDoctor1 and consultantDoctor1; doctors not containing juniorDoctor2.	<i>Expected result</i> false is returned.
Class Team – Test 3 Test for a consultant doctor who is in the team	
<i>Method</i> contains(aDoctor)	<i>Receiver and message</i> team1.contains(consultantDoctor1)
<i>Fixture and input</i> A JuniorDoctor object, juniorDoctor1, with grade = Grade.ONE. A ConsultantDoctor object, consultantDoctor1. A Team object team1 with doctors containing juniorDoctor1 and consultantDoctor1.	<i>Expected result</i> true is returned.
Class Team – Test 4 Test for a consultant doctor who is not in the team	
<i>Method</i> contains(aDoctor)	<i>Receiver and message</i> team1.contains(consultantDoctor2)
<i>Fixture and input</i> A JuniorDoctor object, juniorDoctor1, with grade = Grade.ONE. A ConsultantDoctor object, consultantDoctor1. A ConsultantDoctor object, consultantDoctor2. A Team object team1 with: doctors containing juniorDoctor1 and consultantDoctor1; doctors not containing consultantDoctor2.	<i>Expected result</i> false is returned.

In Exercise 13 you will code and test the `Team` method `contains(aDoctor)`. You will be provided with a partially implemented version of the Hospital core system, `Hospital_Ex_13`. This builds on `Hospital_Ex_9`, which you examined in Section 4. It contains an implementation of the *Admit Patient* use case, the `Team` method `getPatientsAndWards()` and the tests for this method. However, so that you are not distracted by a proliferation of test methods, tests for the methods of *Admit Patient* are absent. Therefore, unlike the testing of a real system, when you start to implement the *Treat Patient* use case in `Hospital_Ex_13`, the regression tests will simply be the tests for `Team`'s `getPatientsAndWards()` method. Although the tests you will be running are not a complete set, they are indicative of what is involved in regression testing.

Exercise 13



When carrying out this exercise you may need to refer again to Section 6 of the *NetBeans Guide*. Unlike the examples in the *NetBeans Guide*, we do not initially create an empty version of the method under test. We will not explain the reasons for this; suffice to say this means that the test class does not automatically create a test method corresponding to the method under test, so you will have to add the test methods manually.

In NetBeans open the project `Hospital_Ex_13`, which is in the folder `M256\M256Code\Exercises\Unit10`.

- (a) A test method corresponding to the test case specified for Test 1 in Table 3 above has been provided in the test class `TeamTest`, which is in the package `hospitalcore` in Test Packages. Examine this method. (It is the final one in the test class, after the test methods for `getPatientsAndWards()`.)

Next, in the test class `TeamTest`, implement test methods corresponding to the test case specifications for Tests 2, 3 and 4 in Table 4.

- (b) In the class `Team`, implement the method `contains(aDoctor)` according to the specification below. (Remember that a `Team` object's `doctors` instance variable references the `Doctor` objects linked to that `Team` object.)

```
boolean contains(Doctor aDoctor)
```

Post-condition: returns `true` if `aDoctor` is linked to the receiver, `false` otherwise.

- (c) Now execute the test runner on the project `Hospital_Ex_13`. This will execute the entire suite of tests, that is, all the tests for both `getPatientsAndWards()` and `contains(aDoctor)` in `TeamTest`.

- (d) Finally, you will simulate what happens when a method is incorrectly coded. Comment out the method code you wrote in part (b) for `contains(aDoctor)` (that is, prefix the line of code with `//`), and enter the following code:

```
return true;
```

Which of Tests 1, 2, 3 and 4 would you expect to fail?

Execute the test runner on the project `Hospital_Ex_13`. Does the test report confirm your expectations?

Discussion.....

- (a) Examine the test methods written by the course team in the project `Hospital_Ex_13_Sol`, which is in the folder `M256\M256Code\Exercises\Unit10`. (You will find them in the class `TeamTest`, which is in the package `hospitalcore` in Test Packages.)
- (b) An appropriate implementation for `contains(aDoctor)` is also given in the project `Hospital_Ex_13_Sol`. (Look in the class `Team` in the source package `hospitalcore`.)

- (c) You should find that all the tests are passed. If not, then compare your answers to parts (a) and (b) with the project `Hospital_Ex_13_Sol`, as above.
- (d) Tests 2 and 4 should fail in this situation, since the expected result in each case is that the invocation of the `contains(aDoctor)` method returns `false`. The test report should confirm that these two tests have failed.

6.3 Coding and testing the `recordTreatmentBy(aDoctor)` method

Having coded and tested the `contains(aDoctor)` method, we move on to the next method in the implementation schedule, which is the `recordTreatmentBy(aDoctor)` method in `Patient`.

Here is the specification of this method:

```
void recordTreatmentBy(Doctor aDoctor)
```

Pre-condition: `aDoctor` and the receiver must be linked to the same `Team` object; if not, an exception is thrown.

Post-condition: a reference to `aDoctor` is recorded.

This specification is different from those you have looked at so far in this unit, in that it contains an explicit pre-condition. A client of the method is expected to conform to the pre-condition, but if it does not, then, as the specification states, an exception should be thrown. Testing should cover both of the following situations:

- ▶ the `Doctor` object and the receiver `Patient` object are linked to the same `Team` object;
- ▶ the `Doctor` object and the receiver `Patient` object are not linked to the same `Team` object.

These tests should be performed where:

- ▶ the `Doctor` object is a `JuniorDoctor` object;
- ▶ the `Doctor` object is a `ConsultantDoctor` object.

Note that there is nothing in the specification to prevent the `Doctor` object having already treated the `Patient` object, that is, to prevent there already being a reference to `aDoctor` in the `Patient` object. In such a situation, invoking the method `recordTreatmentBy(aDoctor)` should have no effect; in particular, the reference in the `Patient` object should remain unchanged. This should also be tested.

Finally, testing should be performed in situations where the `Patient` object has no previous treatments recorded, and situations where the `Patient` object does have previous treatments recorded.

For this method, then, there are a number of tests corresponding to the various combinations of the situations described above. We will concentrate on just two of them. The ones chosen have no special significance: they are simply samples of appropriate tests.

This pre-condition was incorporated into this method in *Unit 7*, Subsection 3.1.

Exercise 14

Specify a test case for each of the following situations (using a tabular format like that in Table 4):

- (a) the `JuniorDoctor` object and the receiver `Patient` object are linked to the same `Team` object, and the `Patient` object has no previous treatments recorded;
- (b) the `JuniorDoctor` object and the receiver `Patient` object are not linked to the same `Team` object and the `Patient` object has no previous treatments recorded.

Discussion.....

Table 5 shows our specifications.

Table 5 Specifying some test cases for `recordTreatmentBy(aDoctor)`

(a) Class Patient – Test 1 Test for a patient with no previous treatments, and a junior doctor in the right team	
<i>Method</i> <code>recordTreatmentBy(aDoctor)</code>	<i>Receiver and message</i> <code>patient1.recordTreatmentBy(juniorDoctor1)</code>
<i>Fixture and input</i> A <code>Team</code> object <code>team1</code> with: <code>patients</code> containing <code>patient1</code> ; <code>doctors</code> containing <code>juniorDoctor1</code> . A <code>JuniorDoctor</code> object <code>juniorDoctor1</code> . A <code>Patient</code> object <code>patient1</code> with <code>team = team1</code> and <code>doctors</code> empty.	<i>Expected result</i> <code>patient1</code> records a reference to <code>juniorDoctor</code> and no exception is raised.
(b) Class Patient – Test 2 Test for a patient with no previous treatments, and a junior doctor not in the right team	
<i>Method</i> <code>recordTreatmentBy(aDoctor)</code>	<i>Receiver and message</i> <code>patient1.recordTreatmentBy(juniorDoctor2)</code>
<i>Fixture and input</i> A <code>Team</code> object <code>team1</code> with: <code>patients</code> containing <code>patient1</code> ; <code>doctors</code> not containing <code>juniorDoctor2</code> . A <code>JuniorDoctor</code> object <code>juniorDoctor2</code> . A <code>Patient</code> object <code>patient1</code> with <code>team = team1</code> and <code>doctors</code> empty.	<i>Expected result</i> An exception is raised and in <code>patient1</code> <code>doctors</code> is still empty.

In the next exercise you will examine a test method corresponding to Test 2, and then write a test method for Test 1 yourself. You will then implement the method `recordTreatmentBy(aDoctor)` and test it.



Exercise 15

In NetBeans open the project `Hospital_Ex_15`, which is in the folder `M256\M256Code\Exercises\Unit10`.

This project builds on the implementation of the Hospital core system in `Hospital_Ex_13_Sol`.

Do the following:

- 1 Create a test class for the class `Patient`. This will automatically be positioned in the package `hospitalcore` in Test Packages, and will be called `PatientTest`.
 - 2 Add the following import statement to `PatientTest`:


```
import java.text.*;
```

 (This package is required for date formatting.)
 - 3 Delete the following methods from this class:


```
setUp()
tearDown()
testGetName()
testGetSex()
testGetAge()
testGetDateOfBirth()
testToString()
testGetWard()
testAdmit()
testGetDoctors()
```
- (a) A copy of a test method for Test 2 is provided in the file `test2RecordTreatmentBy.txt`, in the folder `M256\M256Code\Exercises\Unit10\Hospital_Ex_15`. Copy this method, `test2RecordTreatmentBy()`, from the text file into the test class `PatientTest`. Examine the code for this method.
- (i) What does the invocation of the `Patient` method `admit(aWard, aTeam)` achieve?
 - (ii) Why is the following code enclosed in a `try` block?


```
patient1.recordTreatmentBy(juniorDoctor2);
```
- (b) Implement, in `PatientTest`, a test method `test1RecordTreatmentBy()` for Test 1. (Your test method will require the `Patient` method `getDoctors()`, which you will find has already been implemented.)
- (c) Implement the method `recordTreatmentBy(aDoctor)` in the class `Patient`, according to the method specification below.

```
void recordTreatmentBy(Doctor aDoctor)
```

Pre-condition: `aDoctor` and the receiver must be linked to the same `Team` object; if not, an exception is thrown.

Post-condition: a reference to `aDoctor` is recorded.

Note:

- ▶ You will need to refer to Figures 10 and 12 in the *Handbook*.
- ▶ Your method should throw an `IllegalArgumentException` if the pre-condition is not met.

Subsection 6.1 of the NetBeans Guide explains how to create a test class.

Now execute the test runner on the project `Hospital_Ex_15`. This will execute the entire suite of tests, that is, all the tests in `TeamTest` and `PatientTest`. Thus not only will the newly added method `recordTreatmentBy(aDoctor)` be tested, but regression testing of those parts of the system already implemented will also be carried out (at least in theory: remember we are not working with a complete set of tests).

If any tests fail, return to your code and fix the problem. Continue testing and fixing until all the tests are passed.

Discussion.....

(a) Regarding the method `test2RecordTreatmentBy()`:

- (i) The method `admit(aWard, aTeam)` is invoked to add `patient1` to the collection of `Patient` objects referenced by `team1`'s instance variable `patients`, and to set `patient1`'s instance variable `team` to reference `team1`.
- (ii) The method `recordTreatmentBy(aDoctor)` (the method under test) is specified to throw an exception in a particular circumstance. By placing the code in which the test message is sent within a `try` block, if an exception is thrown (as would be the correct result for Test 2), the remainder of the `try` block statements are ignored, the `catch` block statements (there are none) are executed and the test should be reported as being passed.

If (incorrectly for Test 2) an exception is not thrown, then the subsequent `try` block statement, `fail("...")`, is executed, resulting in failure of the test.

- (b) This test method is contained in the project `Hospital_Ex_15_Sol`. (You will find it in the class `PatientTest`, which is in the package `hospitalcore` in Test Packages).
- (c) Having coded your method, you should find that all the tests are passed. If not, then use the test report to try to pinpoint what is wrong, and fix it. If you have difficulty, compare your method with that given in the project `Hospital_Ex_15_Sol`.

6.4 Coding the `recordTreatment(aPatient, aDoctor)` method

You have now encountered several test methods, and you have experienced the cycles of coding, testing and fixing that are common to test-driven development. Even with our simple methods, and even though we are restricting the extent of the testing undertaken, the test methods involved have become rather complex, requiring some time and effort to set up and run.

You may be wondering whether the time and effort that goes into testing is worthwhile! But consider the fact that large-scale software can consist of hundreds of components, involving thousands of methods. Certainly, many of these will not be simple to implement, and testing is absolutely essential.

That said, in the next exercise we will leave aside the issues of testing, giving you some more practice in simply coding methods, specifically, the `recordTreatment(aPatient, aDoctor)` method, which is the last in the list of methods identified in Subsection 6.1, required to implement the *Treat Patient* use case. Note that this exercise does not mean that test-driven development has been abandoned; rather, we want you to gain more practice in writing methods. So for this exercise imagine that the tests have already been written, and that you have now progressed to the stage of writing the method code.

Exercise 16

Complete the implementation of the *Treat Patient* use case by implementing `HospCoord`'s coordinating method for this use case, `recordTreatment(aPatient, aDoctor)`, whose specification is as follows.

```
public void recordTreatment(Patient aPatient, Doctor aDoctor)
```

Pre-condition: `aDoctor` and `aPatient` must be linked to the same `Team` object; if not, an exception is thrown.

Post-condition: `aDoctor` is linked to `aPatient`.

You will need to refer to Figures 10 and 12 in the *Handbook*.

You need not include a method comment.

Discussion.....

Here is a suitable implementation.

```
public void recordTreatment(Patient aPatient, Doctor aDoctor)
{
    aPatient.recordTreatmentBy(aDoctor);
}
```

In this section you have applied what you have learnt about coding and testing to the methods required for a particular use case.

7 The Hospital core system implementation

In this final, very short section, we ask you to do an exercise that examines a complete implementation of the core system for the Hospital System. Not only will this round off the unit's introduction to implementation, but it will also provide us with an opportunity to point out a few things that have not arisen in the preceding sections.

However, note that you should not have to spend more than 10 minutes on this exercise.

Exercise 17

In NetBeans open the project `Hospital_Ex_17`, which is in the folder `M256\M256Code\Exercises\Unit10`.

This project builds on the preceding projects in this unit and contains:

- ▶ the implementation of the basic structure of each class, as in Exercise 4;
- ▶ a complete implementation of all the use cases of the Hospital System for which designs were created in earlier units of the course;
- ▶ the test classes that have been created in this unit;
- ▶ implementations of the methods that were introduced (in *Unit 7*) to enable the user interface to supply appropriate arguments to the coordinating messages it sends – methods such as `getWards()` in `HospCoord` (such methods might be implemented as a final stage of implementing the core system).

Look at the classes in the package `hospitalcore`, and for each class briefly confirm the correspondence of the methods that have been implemented with the methods in the class description in the implementation model (Section 5 in the *Handbook*). However, there are some class members (the `HospCoord` static method, `getHospital()`, for example) which do not appear in the class descriptions. Do not be concerned with such members: they are present, for example, to enable the Hospital System to be initiated complete with sample `Ward` and `Team` objects, etc. As mentioned earlier, a full-scale system would have use cases for dealing with the creation of a new ward in the hospital, the formation of a team, etc.

We now ask you to look at two methods of particular interest.

- (a) Examine the implementation of the method `getWards()` in `HospCoord`. Why is the static method `unmodifiableCollection(aCollection)` of the `Collections` class used in this method's code?
- (b) Examine the implementation of the `Patient` method `getConsultantDoctor()`. The specification of this method in the implementation model is as follows:

```
ConsultantDoctor getConsultantDoctor()
```

Post-condition: returns the linked `ConsultantDoctor` object.

Explain how the method corresponds to this specification.



There were two use cases that were not considered in the units at the design stage: *Discharge Patient* and *Transfer Patient*. Of these, *Discharge Patient* has been implemented; *Transfer Patient* has not.

You will learn more about this method in *Unit 14*.

Discussion.....

- (a) In *Unit 5*, Section 6, it was shown that whenever an instance variable references a mutable object, there is a danger in having a public getter method returning the value of that instance variable. A client could use it to create a reference to the mutable object, and inappropriately change its state.

Here there is such a danger with the method `getWards()`. The instance variable `wards` references an instance of the class `HashSet<Ward>`. `HashSet` objects are mutable: their states can be changed by the methods `add(anElement)`, `remove(anElement)` and `clear()`.

In *Unit 5*, Subsection 6.2, it was demonstrated that one way of avoiding problems when an instance variable references a mutable collection is to use the static method `unmodifiableCollection(aCollection)` of the `Collections` class, to create an 'unmodifiable view' of a collection `aCollection`. That is, this static method returns a version of `aCollection` which allows only read-only access.

Notice that other methods of `HospCoord` use this static method for a slightly different, but related, reason. For example, the method `getDoctors(aPatient)` does not return information about an instance variable of `HospCoord`, but it does return information about the value of the instance variable `doctors` of a `Patient` object. Since `doctors` is mutable, referencing an instance of class `HashSet<Doctor>`, the `HospCoord` method `getDoctors(aPatient)` protects it by returning an unmodifiable view of it.

- (b) The method `getConsultantDoctor()` is implemented as follows.

```
/**
 * Returns the consultant doctor responsible for this patient.
 *
 * @return the linked ConsultantDoctor object
 */
ConsultantDoctor getConsultantDoctor()
{
    return team.getConsultantDoctor();
}
```

The expression `return team.getConsultantDoctor()` derives the linked `ConsultantDoctor` object using invariant 6 of the Hospital System, which reads as follows:

The `ConsultantDoctor` object linked to any `Patient` object via `isResponsibleFor` is linked via `isHeadedBy` to the `Team` object to which that `Patient` object is linked.

Thus the method accesses and returns the linked `ConsultantDoctor` object, as required by the method specification.

8

Summary

In this unit you met the idea of test-driven development, in which coding and testing are intertwined, with development proceeding in iterative code–test cycles. You also learnt what is involved in a use case-driven approach to coding and testing, thereby maintaining M256's focus on use cases, a focus typical of an object-oriented approach to software development.

The implementation model forms the basis for tackling one use case at a time, with the dynamic model or models for a use case showing how each of the methods involved is related to the others, and the specification for each method describing what that method should achieve. Having also used the dynamic models to determine a sensible order of implementation, each method can be coded.

Method specifications also provided the basis for specifying test cases for each method. You wrote and ran the corresponding tests using JUnit, the facilities of which also support regression testing, a key feature of test-driven development.

In the final section of the unit you examined parts of a complete implementation of the Hospital core system. This piece of software is the culmination of the development so far in the course. If you looked at the code for this system afresh, knowing nothing about its development, you would probably feel rather daunted by the number of classes and methods, and the complexity of some of them. We hope that this heightens your appreciation of the value of following a systematic approach to software development.

However, the story does not end here. This system is useless without a user interface! User interfaces, in their various guises, are the subject of the next three units. Once a user interface is in place, the acceptance tests that were referred to at the beginning of this unit can be performed by the client.

LEARNING OUTCOMES

After studying this unit you should be able to:

- ▶ describe and use each of this unit's key terms (summarised in the Glossary);
- ▶ understand the aims of, and be able to categorise, different types of testing (e.g. verification, validation, black box testing);
- ▶ use an implementation model as a basis for a first code iteration;
- ▶ identify the correspondence between method code and (a) the method's specification and (b) the relevant walk-through(s) and sequence diagram(s);
- ▶ specify a basic set of test cases for a method based on its specification;
- ▶ write and run a test, based on a test case specification, using JUnit appropriately;
- ▶ identify a suitable order in which to implement the methods required for a use case;
- ▶ implement and test the methods required for a use case, using JUnit appropriately.

Glossary

assertion A JUnit test statement.

black box testing Testing in which a method is tested against its specification, without any examination of code.

block tag A part of a doc comment which contains information about particular elements of a method, such as return values, parameters, etc.

boundary A point at which the software's specified behaviour changes.

bug A defect in software.

first code iteration The first stage in implementing a core system. In M256, this stage implements the basic structure of each class, i.e. its attributes, link references, constructors, attribute getter methods and `toString()` method.

fixture The state of the relevant parts of the system before the particular test.

integration testing Testing that problems do not arise from the interaction of a system's components.

regression testing Rerunning previous tests to check that a change or addition made to one aspect of a system has not damaged another, previously working aspect.

system testing Testing the complete system against its specification once all the components are integrated, including testing aspects of the system such as performance, security, system start up, etc.

test In M256, the activity of checking that a specific aspect of the code is consistent with the implementation model, and hence that it correctly represents the designs.

test case A specification of the data to be used in a particular test, and the expected result.

test class A class in which are implemented the test methods for another class.

test method A method which tests another method.

test runner A feature of JUnit which enables the automatic execution of test methods and the reporting of results.

test suite A grouping of test classes.

test-driven development An approach to coding and testing in which tests are written based on the specification of the aspect of the system under test, before the code is written.

unit testing Testing individual units of software in the system: in an object-oriented approach, units may be considered to be methods, or classes.

use case-driven An approach to implementation in which each use case is tackled one by one: having coded and tested the methods involved in one use case, another use case is tackled.

validation Checking that all aspects of the system and its project documentation are consistent with the customer's requirements. In other words, has the right system been built?

verification Checking that all aspects of the system and its project documentation are consistent and complete with respect to each other. In other words, has the system been built right?

white box testing Testing which involves examination of code.

References

Sink, E. (2006) 'Why we all sell code with bugs', *The Guardian*, 25 May. Available from <http://technology.guardian.co.uk/weekly/story/0,,1781895,00.html> (Accessed 18 December 2006).

Acknowledgements

Grateful acknowledgement is made to the following source.

Subsection 2.2: Sink E., (2006), 'Why We Sell Code with Bugs', *Technology Weekly*, The Guardian Newspapers Limited.

Index

A

acceptance test 8
alternative scenario 23
assertion 35

B

black box testing 10
block tag 18
boundary 11
bug 9

C

coordinating
 message 23
 object 23

D

doc comment 18
dynamic model 6, 23

F

first code iteration 14
fixture 9

I

implementation model 5–6, 8, 14, 23
initial structural model 5
integration testing 12

J

Javadoc 18, 21
JUnit 12–13, 34

M

mutable object 50

O

order of implementation 26, 40

R

regression testing 13

S

scenario 23
sequence diagram 23
stub 40
system testing 13

T

test 8
 case 9
 class 12
 method 12
 runner 34
 suite 13
test-driven development 6, 9, 23

testing

 black box 10
 integration 12
 regression 13
 system 13
 unit 12
 white box 10

toString() 20

typical scenario 23

U

unit testing 12
use case 6
use case-driven 13, 23

V

validation 8
verification 8

W

walk-through 23
white box testing 10

